



INTRODUCTION TO PYTHON

OVERVIEW OF PYTHON

- **High-Level Language:** Python is a high-level, interpreted programming language known for its simplicity and readability.
- **Development History:** Developed by Guido van Rossum and first released in 1991.
- **Popularity and Versatility:** Python has gained popularity due to its versatility and extensive support from a rich ecosystem of libraries and frameworks.
- **Object-Oriented:** It is categorized as an object-oriented language, promoting modular and reusable code.
- **Accessibility:** Python is an excellent choice for both novice programmers and experienced developers.
- **Design Philosophy:** The design philosophy emphasizes code readability and simplicity, allowing for clear and logical code.
- **Collaboration:** This focus on readability fosters better communication and collaboration within development teams, important in professional settings and internships.
- **Dynamic Typing and Memory Management:** Python features dynamic typing and automatic memory management, enabling developers to concentrate on problem-solving rather than complex syntax.
- **Supports Multiple Paradigms:** The language supports procedural, object-oriented, and functional programming paradigms, offering flexibility.
- **Wide Application Range:** Python is utilized in various applications, including web development, scientific computing, data analysis, machine learning, and automation.
- **Industry Relevance:** Its combination of features makes Python a language of choice for many tech companies and academic institutions.
- **Skill Value:** Proficiency in Python is a valuable skill for interns entering the job market.

IMPORTANCE OF PYTHON

- **Preferred Language:** Python has emerged as the preferred programming language for numerous developers and organizations due to its adaptability and extensive ecosystem.
- **Versatile Tool:** It is utilized across various domains, including web development, data science, artificial intelligence, automation, and software development, making it an essential tool in a developer's toolkit.
- **Strong Community Support:** The Python Software Foundation and a vibrant global community contribute to the language's growth by providing resources, documentation, and libraries that enhance its capabilities.
- **Collaborative Environment:** This supportive ecosystem fosters innovation, allowing developers to

share knowledge and solutions, benefiting interns who can leverage these resources during their learning process.

- **Standard in Emerging Technologies:** Python is a standard language in data science and machine learning, supported by powerful libraries such as NumPy, Pandas, TensorFlow, and scikit-learn.
- **Data Manipulation and Analysis:** These libraries facilitate data manipulation, analysis, and model building, making Python crucial for data-driven projects.
- **Job Market Advantage:** Interns proficient in Python and its libraries have a significant advantage in the job market, enabling them to contribute meaningfully to projects from the start.
- **Educational Appeal:** Python's ease of use and rapid learning curve make it attractive for educational institutions, preparing new generations of programmers with relevant skills for modern technology.

APPLICATIONS OF PYTHON

- **Web Development:** Frameworks like Django and Flask enable developers to create robust and scalable web applications quickly, making Python relevant for interns in tech companies focused on quick prototyping and agile development.
- **Practical Skills for Interns:** Learning these web frameworks provides interns with practical skills that are immediately applicable in the workplace.
- **Data Science Applications:** Python's powerful libraries support extensive data analysis, visualization, and machine learning, making it essential for data-centric roles.
- **Data Visualization Tools:** Libraries such as Matplotlib and Seaborn facilitate sophisticated data visualization, while Pandas is crucial for data manipulation.
- **Insight Extraction:** Interns can leverage these tools to extract insights from complex datasets, contributing to informed decision-making processes within their organizations.
- **Automation and Scripting:** Python is extensively used for automation and scripting tasks, allowing developers to automate repetitive processes.
- **Efficiency in Business:** The ability to write Python scripts for automation is invaluable in business environments, enhancing efficiency and saving time.
- **Research and Scientific Computing:** Libraries like SciPy assist with complex calculations, and Jupyter Notebooks provide an interactive environment for documenting code and results.
- **Collaboration in Research:** This integration is particularly useful for interns in research or academic settings, where clear and interactive presentation of findings is essential.
- **Artificial Intelligence and Machine Learning:** Python serves as the backbone for many machine learning frameworks and tools, providing resources for building intelligent systems.
- **Skills for Aspiring AI Interns:** Interns aiming for AI and machine learning roles can benefit significantly from Python skills, as they are often required to implement algorithms and analyze data.
- **User-Friendly Language:** Python stands out as a versatile, powerful, and user-friendly language, establishing itself as a cornerstone in various technological fields.
- **Extensive Community Support:** Its broad application spectrum and adaptability make Python an ideal choice for both novice and experienced developers.
- **Enhancing Employability:** Proficiency in Python enhances the technical skill set of interns and significantly improves their employability and readiness to contribute to diverse projects.
- **Growing Demand:** As industries increasingly focus on data-driven decision-making and automation, the demand for Python developers is expected to continue rising, making it vital to master for future success.

INSTALLATION AND ENVIRONMENT SETUP

INSTALLING PYTHON

1. Download Python:

- Visit the [official Python website](#).
- Choose the appropriate installer for your operating system (Windows, macOS, or Linux).

2. Run the Installer:

- Locate the downloaded installer file and run it.
- For Windows, double-click the `.exe` file; for macOS, open the `.pkg` file; for Linux, use the terminal.

3. Installation Setup:

- For Windows, check the box to "Add Python to PATH" before clicking "Install Now."
- Choose "Customize installation" for additional options if desired.

4. Follow Installation Prompts:

- Click "Next" or "Continue" to proceed through the installation prompts.
- Review and accept the license agreement if prompted.

5. Complete Installation:

- Wait for the installation to finish.
- Click "Finish" to close the installer.

6. Verify Installation:

- Open a command prompt (Windows) or terminal (macOS/Linux).
- Type `python --version` or `python3 --version` to check if Python is installed correctly.

7. Install pip (if necessary):

- Pip is included with recent versions of Python, but if you need to install it manually, follow the instructions from the official documentation.

8. Update Python (Optional):

- After installation, you may want to check for updates or install the latest version as needed.

INSTALLING ANACONDA

1. Download Anaconda:

- Visit the [Anaconda website](#).
- Choose the appropriate installer for your operating system (Windows, macOS, or Linux).

2. Run the Installer:

- Locate the downloaded installer file and run it.
- For Windows, double-click the `.exe` file; for macOS, open the `.pkg` file; for Linux, use the terminal with the `.sh` file.

3. Follow Installation Prompts:

- Click "Next" or "Continue" to proceed with the installation.
- Review and accept the license agreement.

4. Choose Installation Type:

- Select whether to install for "Just Me" or "All Users" (Windows).
- Choose the installation location (default is usually fine).

5. Advanced Options:

- (Optional) You can choose to add Anaconda to your PATH variable (not recommended for all users) or register Anaconda as the default Python (recommended).

6. Complete Installation:

- Click "Install" and wait for the installation process to complete.
- Once finished, click "Next" and then "Finish."

7. Verify Installation:

- Open the Anaconda Navigator or command prompt/terminal and type `conda --version` to verify the installation.

8. Update Anaconda (Optional):

- It's a good practice to update Anaconda after installation. Open Anaconda Prompt or terminal and run `conda update conda`.

INSTALLING SPYDER

1. Install Anaconda (Recommended):

- If you haven't already, download and install Anaconda from the [Anaconda website](#). Spyder comes pre-installed with Anaconda.

2. Open Anaconda Navigator:

- Launch Anaconda Navigator from your applications or programs menu.

3. Install Spyder via Anaconda Navigator:

- In Anaconda Navigator, find Spyder in the list of applications.
- Click the "Install" button next to Spyder.

4. Install via Command Line (If not using Anaconda):

- Open your terminal (Linux/macOS) or command prompt (Windows).
- Ensure that you have Python and pip installed, then run:
`pip install spyder`

5. Verify Installation:

- After installation, you can start Spyder by searching for it in your applications or by typing `spyder` in the command prompt or terminal.

6. Update Spyder (Optional):

- To update Spyder to the latest version, you can use the command:
`conda update spyder`
- Or if you installed via pip:
`pip install --upgrade spyder`

INSTALLING JUPYTER NOTEBOOK

1. Install Anaconda (Recommended):

- If you haven't already, download and install Anaconda from the [Anaconda website](#). Jupyter Notebook is included with the Anaconda distribution.

2. Open Anaconda Navigator:

- Launch Anaconda Navigator from your applications or programs menu.

3. Install Jupyter Notebook via Anaconda Navigator:

- In Anaconda Navigator, find Jupyter Notebook in the list of applications.
- Click the "Install" button next to Jupyter Notebook if it is not already installed.

4. Install via Command Line (If not using Anaconda):

- Open your terminal (Linux/macOS) or command prompt (Windows).
- Ensure that you have Python and pip installed, then run:
`pip install notebook`

5. Verify Installation:

- After installation, you can start Jupyter Notebook by typing the following command in the command prompt or terminal:
`jupyter notebook`

6. Open Jupyter Notebook:

- This command will launch Jupyter Notebook in your default web browser.

7. Update Jupyter Notebook (Optional):

- To update Jupyter Notebook to the latest version, you can use the command:
`conda update notebook`
- Or if you installed via pip:
`pip install --upgrade notebook`

INSTALLING VISUAL STUDIO

1. **Download Visual Studio Code:**
 - Visit the [Visual Studio Code website](#).
 - Click on the "Download" button for your operating system (Windows, macOS, or Linux).
2. **Run the Installer:**
 - Locate the downloaded installer file and run it.
 - For Windows, double-click the `.exe` file; for macOS, open the `.dmg` file; for Linux, follow the installation instructions for your specific distribution.
3. **Follow Installation Prompts:**
 - For Windows, accept the license agreement and select installation options, including creating a desktop icon and adding VS Code to your PATH (recommended).
 - Click "Install" to begin the installation.
4. **Complete Installation:**
 - Wait for the installation to finish.
 - Once complete, click "Finish" to close the installer.
5. **Launch Visual Studio Code:**
 - Open Visual Studio Code from your applications or programs menu.
6. **Install Extensions (Optional):**
 - To enhance functionality, you can install extensions by clicking on the Extensions view icon (or pressing `Ctrl+Shift+X`) and searching for desired extensions (e.g., Python, Jupyter).
7. **Update Visual Studio Code (Optional):**
 - Visual Studio Code updates automatically, but you can manually check for updates by going to the Help menu and selecting "Check for Updates."

VARIABLES AND DATA TYPES

VARIABLES

Variables in Python are used to store data values. Unlike some programming languages, Python does not require explicit declaration of variable types; instead, it infers the type from the assigned value. Variables can hold different types of data throughout their lifecycle. The naming convention for variables allows for a combination of letters, digits, and underscores, but must start with a letter or underscore.

```
In [ ]: # Define initial variables
name = "Catherine Elizabeth Parker"
age = 38                                # Integer
base_salary = 8350.75                  # Float monthly base salary
performance_multiplier = 1.18           # Float performance-based salary multiplier
goal_savings = 200000                  # Integer target savings amount

# Nested functions for complex operations
def calculate_annual_salary(salary, multiplier):
    """Calculate adjusted annual salary based on performance multiplier."""
    return salary * multiplier * 12

def years_to_goal(salary, target, saving_rate=0.25):
    """Calculate years to reach savings goal based on saving a portion of salary."""
    return int(target / (salary * saving_rate))

# Use the functions to compute complex values
annual_salary = calculate_annual_salary(base_salary, performance_multiplier)
target_years = years_to_goal(annual_salary, goal_savings)

# String manipulation to extract initials and rearrange them
initials = "".join([word[0] for word in name.split()])
rearranged_initials = initials[::-1] # Reverse initials for a unique ID

# Generate a salary rating based on the base salary amount
salary_rating = "High" if base_salary > 8000 else "Moderate" if base_salary > 5000 else "Low"

# Calculate a weighted score by combining age and annual salary, demonstrating complex int/float logic
weighted_score = round((age * 0.5 + annual_salary * 0.0001), 2)

# Format the results with advanced string operations
output_message = (
    f"Employee: {name}\n"
    f"Annual Salary: {annual_salary}\n"
    f"Target Years: {target_years}\n"
    f"Salary Rating: {salary_rating}\n"
    f"Weighted Score: {weighted_score}"
)
```

```

f"Initials (Reversed): {rearranged_initials}\n"
f"Age: {age} years (Weighted Score: {weighted_score})\n"
f"Monthly Base Salary: ${base_salary:,.2f}\n"
f"Performance Multiplier: {performance_multiplier:.2f}\n"
f"Adjusted Annual Salary: ${annual_salary:,.2f} ({salary_rating} Salary)\n"
f"Years to Reach Savings Goal of ${goal_savings:,.2f}: "
f"{target_years if target_years > 0 else 'Already achieved!'}\n"
f"Unique Code: {rearranged_initials + str(age % 10) + str(int(base_salary))[4:]}"
)

# Display the output message
print(output_message)

```

```

Employee: Catherine Elizabeth Parker
Initials (Reversed): PEC
Age: 38 years (Weighted Score: 30.82)
Monthly Base Salary: $8,350.75
Performance Multiplier: 1.18
Adjusted Annual Salary: $118,246.62 (High Salary)
Years to Reach Savings Goal of $200,000.00: 6
Unique Code: PEC88350

```

This program calculates an employee's adjusted annual salary using a performance multiplier and estimates the years required to reach a savings goal based on a portion of the annual salary. It combines string manipulation to extract and reverse initials, while also generating a unique code based on personal and financial data. The program incorporates complex conditional logic, weighted scoring, and advanced formatting to present a detailed financial summary and personalized report.

DEFINING A VARIABLE

Defining a variable means creating a named storage location in a program to hold a value that can be accessed and modified. This variable can store various data types, such as numbers or text, allowing flexible, reusable code.

```

In [ ]: # Define variables
name = "Alice"
age = 25

# Print a greeting using the variables
print("Hello, " + name + "! You are " + str(age) + " years old.")

```

```
Hello, Alice! You are 25 years old.
```

DATA TYPES

Data types in Python determine the kind of operations that can be performed on the data and the amount of memory allocated. Python has several built-in data types including integers, floats, strings, lists, tuples, sets, and dictionaries. Each of these data types has its own characteristics and methods.

```

In [ ]: # Multiple assignments in one line
x, y, z = 1, 2.5, "Hello"

# Dynamically typing variables
print("Original values:")
print(f"x: {x} (Type: {type(x)})")
print(f"y: {y} (Type: {type(y)})")
print(f"z: {z} (Type: {type(z)})")

# Reassigning variables with different types
x = "New Value"
y = [1, 2, 3]
z = {1: "one", 2: "two"}

print("\nAfter reassigning:")
print(f"x: {x} (Type: {type(x)})")
print(f"y: {y} (Type: {type(y)})")
print(f"z: {z} (Type: {type(z)})")

```

```

Original values:
x: 1 (Type: <class 'int'>)
y: 2.5 (Type: <class 'float'>)
z: Hello (Type: <class 'str'>)

```

```

After reassigning:
x: New Value (Type: <class 'str'>)
y: [1, 2, 3] (Type: <class 'list'>)
z: {1: 'one', 2: 'two'} (Type: <class 'dict'>)

```

INTEGERS

Integers are whole numbers that can be either positive or negative, and they do not contain any decimal points. They can represent a wide range of values, including counts, quantities, identifiers, and more. In Python, integers can be of arbitrary length, which means they can grow as large as the memory allows, making them suitable for calculations requiring high precision or large datasets. Python supports several built-in operations for integers, including arithmetic operations (addition, subtraction, multiplication, division), comparison operations, and bitwise operations.

```
In [ ]: # Integer operations showcasing various types of integers
# Positive and Negative Integers
positive_int = 15
negative_int = -8
zero_int = 0

# Performing arithmetic operations
addition = positive_int + negative_int      # 15 + (-8)
subtraction = positive_int - negative_int    # 15 - (-8)
multiplication = positive_int * 2           # 15 * 2
division = positive_int / 4                 # Integer division
modulus = positive_int % 4                  # Remainder of 15 / 4
exponentiation = positive_int ** 2          # 15 raised to the power of 2

# Printing the results
print(f"Positive Integer: {positive_int}")
print(f"Negative Integer: {negative_int}")
print(f"Zero Integer: {zero_int}")
print(f"Addition: {positive_int} + {negative_int} = {addition}")
print(f"Subtraction: {positive_int} - {negative_int} = {subtraction}")
print(f"Multiplication: {positive_int} * 2 = {multiplication}")
print(f"Integer Division: {positive_int} // 4 = {division}")
print(f"Modulus: {positive_int} % 4 = {modulus}")
print(f"Exponentiation: {positive_int} ** 2 = {exponentiation}")
```

```
Positive Integer: 15
Negative Integer: -8
Zero Integer: 0
Addition: 15 + -8 = 7
Subtraction: 15 - -8 = 23
Multiplication: 15 * 2 = 30
Integer Division: 15 // 4 = 3
Modulus: 15 % 4 = 3
Exponentiation: 15 ** 2 = 225
```

FLOAT

Floats, or floating-point numbers, are numeric data types in Python that represent real numbers, allowing for a fractional part. They can be positive or negative and are typically used when more precision is required than what integers can provide, especially in scientific calculations, financial applications, and anywhere that requires decimal representation. In Python, floats are represented using a decimal point or in exponential (scientific) notation, and they adhere to the IEEE 754 standard for floating-point arithmetic. Python handles float precision, but it's important to note that floating-point arithmetic can introduce rounding errors due to the binary representation of decimal numbers.

```
In [ ]: # Floating-point operations showcasing various types of floats
# Positive and Negative Floats
positive_float = 3.14
negative_float = -2.71
zero_float = 0.0
exponential_float_positive = 1.5e2 # Equivalent to 150.0
exponential_float_negative = 2.5e-3 # Equivalent to 0.0025

# Performing arithmetic operations
addition = positive_float + negative_float      # 3.14 + (-2.71)
subtraction = positive_float - negative_float    # 3.14 - (-2.71)
multiplication = positive_float * 2.5           # 3.14 * 2.5
division = positive_float / 2                   # 3.14 / 2
modulus = positive_float % 1.5                  # Remainder of 3.14 / 1.5
exponentiation = positive_float ** 2            # 3.14 raised to the power of 2

# Printing the results
print(f"Positive Float: {positive_float}")
print(f"Negative Float: {negative_float}")
print(f"Zero Float: {zero_float}")
print(f"Exponential Float (Positive): {exponential_float_positive}")
print(f"Exponential Float (Negative): {exponential_float_negative}")
print(f"Addition: {positive_float} + {negative_float} = {addition}")
```

```
print(f"Subtraction: {positive_float} - {negative_float} = {subtraction}")
print(f"Multiplication: {positive_float} * 2.5 = {multiplication}")
print(f"Division: {positive_float} / 2 = {division}")
print(f"Modulus: {positive_float} % 1.5 = {modulus}")
print(f"Exponentiation: {positive_float} ** 2 = {exponentiation}")
```

```
Positive Float: 3.14
Negative Float: -2.71
Zero Float: 0.0
Exponential Float (Positive): 150.0
Exponential Float (Negative): 0.0025
Addition: 3.14 + -2.71 = 0.43000000000000016
Subtraction: 3.14 - -2.71 = 5.85
Multiplication: 3.14 * 2.5 = 7.8500000000000005
Division: 3.14 / 2 = 1.57
Modulus: 3.14 % 1.5 = 0.14000000000000012
Exponentiation: 3.14 ** 2 = 9.8596
```

STRINGS

Strings in Python are sequences of characters used to represent text data. They are one of the most versatile data types in Python and can encompass letters, numbers, symbols, and spaces. Strings are mutable, meaning once created, their values can be changed. However, various string methods allow for manipulation and formatting, making them a powerful tool for handling textual information. Strings can be defined using single quotes (' '), double quotes (" "), or triple quotes (''' ''' or "" "" """) for multi-line strings. They can also include escape characters, such as `\n` for newline and `\t` for tab, allowing for formatted output.

```
In [ ]: # String operations showcasing various types of strings
# Single and Double-quoted Strings
single_quoted_string = 'Hello, World!'
double_quoted_string = "Welcome to Python programming."

# Multi-line String
multi_line_string = '''This is a multi-line string.
It can span multiple lines and preserve formatting.'''

# Raw String
raw_string = r"C:\Users\Name\Documents\Python"

# String Concatenation
greeting = single_quoted_string + " " + double_quoted_string

# String Formatting
formatted_string = f"{single_quoted_string} It's a great day!"

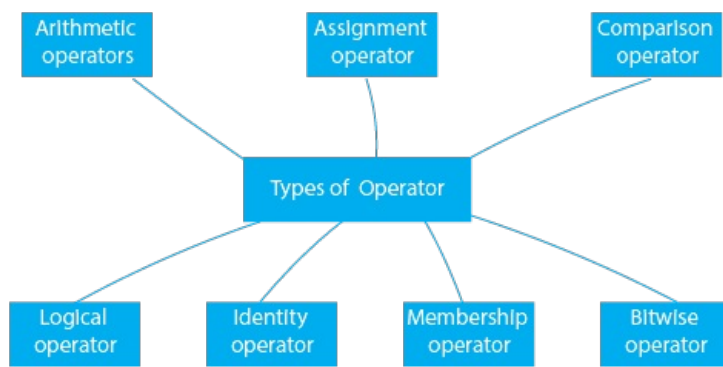
# String Length
length_of_string = len(single_quoted_string)

# Printing the results
print(f"Single-quoted String: {single_quoted_string}")
print(f"Double-quoted String: {double_quoted_string}")
print(f"Multi-line String:\n{multi_line_string}")
print(f"Raw String: {raw_string}")
print(f"Greeting: {greeting}")
print(f"Formatted String: {formatted_string}")
print(f"Length of Single-quoted String: {length_of_string}")
```

```
Single-quoted String: Hello, World!
Double-quoted String: Welcome to Python programming.
Multi-line String:
This is a multi-line string.
It can span multiple lines and preserve formatting.
Raw String: C:\Users\Name\Documents\Python
Greeting: Hello, World! Welcome to Python programming.
Formatted String: Hello, World! It's a great day!
Length of Single-quoted String: 13
```

OPERATORS

Operators in Python are symbols or keywords used to perform operations on variables and values. They allow us to perform computations, compare values, and manipulate data in various ways. Python supports several types of operators, which can be categorized based on their functionality.



```

In [ ]: import random
import time

# Initialize fleet of vehicles with various parameters
vehicles = [
    {'id': 1, 'speed': 50, 'fuel': 80, 'temperature': 22.5, 'weather': 'clear', 'road': 'dry', 'obstacles': False},
    {'id': 2, 'speed': 65, 'fuel': 30, 'temperature': 20.0, 'weather': 'rainy', 'road': 'wet', 'obstacles': True},
    {'id': 3, 'speed': 40, 'fuel': 15, 'temperature': 28.0, 'weather': 'clear', 'road': 'dry', 'obstacles': False},
    {'id': 4, 'speed': 75, 'fuel': 60, 'temperature': 15.0, 'weather': 'snowy', 'road': 'icy', 'obstacles': True}
]

# Define thresholds and constants
fuel_threshold = 20
speed_limit = 100
temperature_threshold = 30
high_traffic_threshold = 4
max_obstacles = 1
emergency_traffic_level = 5
high_risk_weather = ['rainy', 'snowy']
acceptable_weather = ['clear', 'partly cloudy', 'cloudy']
road_conditions = ['dry', 'wet', 'icy']

# Function to simulate vehicle behavior based on conditions
def evaluate_vehicle(vehicle):
    # Arithmetic Operations: Adjust speed and fuel efficiency
    adjusted_speed = (vehicle['speed'] * 1.2) - (vehicle['temperature'] * 0.5)
    adjusted_fuel = vehicle['fuel'] - (vehicle['speed'] * 0.1)

    # Comparison Operations: Evaluate conditions to decide vehicle actions
    speed_within_limit = adjusted_speed <= speed_limit
    fuel_sufficient = adjusted_fuel >= fuel_threshold
    weather_clear = vehicle['weather'] in acceptable_weather
    road_safe = vehicle['road'] in road_conditions
    traffic_clear = vehicle['traffic'] < high_traffic_threshold
    is_obstacle = vehicle['obstacles'] == False

    # Logical Operations: Combine multiple conditions to form a decision rule
    decision = (
        (speed_within_limit and fuel_sufficient) and
        (weather_clear or road_safe) and
        traffic_clear and
        is_obstacle
    )

    # Bitwise Operations: Detect obstacles in sensor data
    if vehicle['obstacles']:
        vehicle['status'] = "obstacle detected"
    elif (vehicle['traffic'] <= 1) & emergency_traffic_level:
        vehicle['status'] = "emergency rerouting"

    # Identity and Membership Operators: Ensure vehicle state is valid and weather conditions are handled
    if vehicle['status'] is not "stopped" and vehicle['weather'] not in high_risk_weather:
        vehicle['status'] = "continue moving"
    else:
        vehicle['status'] = "stop and reassess"

    # Conditional Expression: If conditions are met, vehicle keeps moving, else stops
    final_decision = "Continue Moving" if decision else "Stop Vehicle and Reassess"
    vehicle['final_status'] = final_decision

    # Update vehicle parameters based on conditions
    vehicle['speed'] = adjusted_speed if speed_within_limit else vehicle['speed'] - 10
    vehicle['fuel'] = adjusted_fuel if fuel_sufficient else vehicle['fuel'] - 5

    # Return updated vehicle
    return vehicle
  
```

```

# Simulate fleet operation over time
def fleet_simulation():
    for time_step in range(10): # Simulate for 10 time steps
        print(f"Time Step {time_step + 1}")
        for vehicle in vehicles:
            updated_vehicle = evaluate_vehicle(vehicle)
            print(f"Vehicle {updated_vehicle['id']}: {updated_vehicle['status']} with speed {updated_vehicle['speed']} km/h, fuel {updated_vehicle['fuel']}%")

            # Complex decision making involving multiple operators
            if updated_vehicle['final_status'] == "Stop Vehicle and Reassess":
                if updated_vehicle['fuel'] < fuel_threshold:
                    print(f"Vehicle {updated_vehicle['id']} needs to refuel. Current fuel: {updated_vehicle['fuel']}%")
                if updated_vehicle['weather'] in high_risk_weather:
                    print(f"Vehicle {updated_vehicle['id']} cannot move due to dangerous weather conditions.")
                if updated_vehicle['traffic'] >= high_traffic_threshold:
                    print(f"Vehicle {updated_vehicle['id']} is rerouting due to high traffic.")
            print() # Empty line for readability

# Run the fleet simulation
fleet_simulation()

```

Time Step 1

Vehicle 1: continue moving with speed 48.75 km/h, fuel 75.0%

Vehicle 2: stop and reassess with speed 68.0 km/h, fuel 23.5%

Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed 34.0 km/h, fuel 10%

Vehicle 3 needs to refuel. Current fuel: 10%

Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 82.5 km/h, fuel 52.5%

Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 2

Vehicle 1: continue moving with speed 47.25 km/h, fuel 70.125%

Vehicle 2: stop and reassess with speed 71.6 km/h, fuel 18.5%

Vehicle 2 needs to refuel. Current fuel: 18.5%

Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed 26.799999999999997 km/h, fuel 5%

Vehicle 3 needs to refuel. Current fuel: 5%

Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 91.5 km/h, fuel 44.25%

Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 3

Vehicle 1: continue moving with speed 45.449999999999996 km/h, fuel 65.4%

Vehicle 2: stop and reassess with speed 75.91999999999999 km/h, fuel 13.5%

Vehicle 2 needs to refuel. Current fuel: 13.5%

Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed 18.159999999999997 km/h, fuel 0%

Vehicle 3 needs to refuel. Current fuel: 0%

Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 81.5 km/h, fuel 35.1%

Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 4

Vehicle 1: continue moving with speed 43.28999999999999 km/h, fuel 60.855000000000004%

Vehicle 2: stop and reassess with speed 81.10399999999998 km/h, fuel 8.5%

Vehicle 2 needs to refuel. Current fuel: 8.5%

Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed 7.7919999999999945 km/h, fuel -5%

Vehicle 3 needs to refuel. Current fuel: -5%

Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 90.3 km/h, fuel 26.950000000000003%

Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 5

Vehicle 1: continue moving with speed 40.697999999999986 km/h, fuel 56.526%

Vehicle 2: stop and reassess with speed 87.32479999999998 km/h, fuel 3.5%

Vehicle 2 needs to refuel. Current fuel: 3.5%

Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed -4.649600000000007 km/h, fuel -10%
Vehicle 3 needs to refuel. Current fuel: -10%
Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 80.3 km/h, fuel 21.950000000000003%
Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 6

Vehicle 1: continue moving with speed 37.58759999999998 km/h, fuel 52.4562%

Vehicle 2: stop and reassess with speed 94.78975999999997 km/h, fuel -1.5%
Vehicle 2 needs to refuel. Current fuel: -1.5%
Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed -19.579520000000001 km/h, fuel -15%
Vehicle 3 needs to refuel. Current fuel: -15%
Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 88.86 km/h, fuel 16.950000000000003%
Vehicle 4 needs to refuel. Current fuel: 16.950000000000003%
Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 7

Vehicle 1: continue moving with speed 33.85511999999998 km/h, fuel 48.69744000000001%

Vehicle 2: stop and reassess with speed 84.78975999999997 km/h, fuel -6.5%
Vehicle 2 needs to refuel. Current fuel: -6.5%
Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed -37.495424000000014 km/h, fuel -20%
Vehicle 3 needs to refuel. Current fuel: -20%
Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 99.13199999999999 km/h, fuel 11.950000000000003%
Vehicle 4 needs to refuel. Current fuel: 11.950000000000003%
Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 8

Vehicle 1: continue moving with speed 29.376143999999975 km/h, fuel 45.31192800000001%

Vehicle 2: stop and reassess with speed 91.74771199999996 km/h, fuel -11.5%
Vehicle 2 needs to refuel. Current fuel: -11.5%
Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed -58.994508800000001 km/h, fuel -25%
Vehicle 3 needs to refuel. Current fuel: -25%
Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 89.13199999999999 km/h, fuel 6.950000000000003%
Vehicle 4 needs to refuel. Current fuel: 6.950000000000003%
Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 9

Vehicle 1: continue moving with speed 24.00137279999997 km/h, fuel 42.37431360000001%

Vehicle 2: stop and reassess with speed 81.74771199999996 km/h, fuel -16.5%
Vehicle 2 needs to refuel. Current fuel: -16.5%
Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed -84.793410560000001 km/h, fuel -30%
Vehicle 3 needs to refuel. Current fuel: -30%
Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 99.45839999999998 km/h, fuel 1.950000000000028%
Vehicle 4 needs to refuel. Current fuel: 1.950000000000028%
Vehicle 4 cannot move due to dangerous weather conditions.

Time Step 10

Vehicle 1: continue moving with speed 17.55164735999996 km/h, fuel 39.97417632000001%

Vehicle 2: stop and reassess with speed 88.09725439999995 km/h, fuel -21.5%
Vehicle 2 needs to refuel. Current fuel: -21.5%
Vehicle 2 cannot move due to dangerous weather conditions.

Vehicle 3: continue moving with speed -115.75209267200002 km/h, fuel -35%
Vehicle 3 needs to refuel. Current fuel: -35%
Vehicle 3 is rerouting due to high traffic.

Vehicle 4: stop and reassess with speed 89.45839999999998 km/h, fuel -3.04999999999997%
Vehicle 4 needs to refuel. Current fuel: -3.04999999999997%
Vehicle 4 cannot move due to dangerous weather conditions.

```
<>:52: SyntaxWarning: "is not" with a literal. Did you mean "!="?
<>:52: SyntaxWarning: "is not" with a literal. Did you mean "!="?
<ipython-input-12-62faac066e3d>:52: SyntaxWarning: "is not" with a literal. Did you mean "!="?
    if vehicle['status'] is not "stopped" and vehicle['weather'] not in high_risk_weather:
```

This program simulates an autonomous vehicle fleet making real-time decisions based on complex factors like traffic, fuel levels, weather, and obstacles. It integrates arithmetic, logical, comparison, bitwise, assignment, membership, identity, and conditional operators to dynamically adjust the vehicles' status and actions. The fleet evaluates conditions, reroutes if necessary, and responds to emergencies with intricate decision-making logic, mimicking real-world autonomous driving systems.

ARITHMETIC OPERATOR

Arithmetic operators perform basic mathematical operations like addition, subtraction, multiplication, and division. They work on numerical data types, including integers and floats.

Operators: +, -, *, /, %, **, //

```
In [ ]: a = 15
        b = 4

        # Addition and Subtraction
        sum_result = a + b
        diff_result = a - b

        # Multiplication, Division, Floor Division
        prod_result = a * b
        div_result = a / b
        floor_div_result = a // b

        # Modulus and Exponentiation
        mod_result = a % b
        exp_result = a ** b

        print(f"Sum: {sum_result}, Difference: {diff_result}")
        print(f"Product: {prod_result}, Division: {div_result}, Floor Division: {floor_div_result}")
        print(f"Modulus: {mod_result}, Exponentiation: {exp_result}")
```

```
Sum: 19, Difference: 11
Product: 60, Division: 3.75, Floor Division: 3
Modulus: 3, Exponentiation: 50625
```

COMPARISON OPERATOR

Comparison operators compare two values and return a Boolean result (True or False). These operators are commonly used in conditional statements and loops to control the flow of programs.

Operators: ==, !=, >, <, >=, <=

```
In [ ]: x = 10
        y = 20

        # Equality and Inequality
        print("x == y:", x == y)
        print("x != y:", x != y)

        # Greater than, Less than
        print("x > y:", x > y)
        print("x < y:", x < y)

        # Greater than or equal to, Less than or equal to
        print("x >= y:", x >= y)
        print("x <= y:", x <= y)
```

```
x == y: False
x != y: True
x > y: False
x < y: True
x >= y: False
x <= y: True
```

LOGICAL OPERATORS

Logical operators allow for the combination of multiple conditions, making complex conditions possible. These operators are especially useful in conditional statements where multiple criteria

need to be evaluated.

Operators: and, or, not

```
In [ ]: a = 5
b = 10
c = 15

# Logical AND
print("a < b and b < c:", a < b and b < c)

# Logical OR
print("a > b or b < c:", a > b or b < c)

# Logical NOT
print("not (a < b):", not (a < b))
```

a < b and b < c: True

a > b or b < c: True

not (a < b): False

BITWISE OPERATOR

Bitwise operators work at the binary level, performing operations on the individual bits of numbers. They are primarily used in low-level programming and can be useful in scenarios like encryption, compression, and optimization.

Operators: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift)

```
In [ ]: a = 0b1101 # 13 in binary
b = 0b1011 # 11 in binary

# Bitwise AND, OR, XOR
and_result = a & b
or_result = a | b
xor_result = a ^ b

# Bitwise NOT
not_result = ~a

# Bitwise Left and Right Shift
left_shift = a << 1
right_shift = a >> 1

print(f"Bitwise AND: {bin(and_result)}")
print(f"Bitwise OR: {bin(or_result)}")
print(f"Bitwise XOR: {bin(xor_result)}")
print(f"Bitwise NOT: {bin(not_result)}")
print(f"Left Shift: {bin(left_shift)}, Right Shift: {bin(right_shift)}")
```

Bitwise AND: 0b1001

Bitwise OR: 0b1111

Bitwise XOR: 0b110

Bitwise NOT: -0b1110

Left Shift: 0b11010, Right Shift: 0b110

ASSIGNMENT OPERATOR

Assignment operators assign values to variables. They also support compound assignments, which combine assignment with arithmetic or bitwise operations.

Operators: =, +=, -=, *=, /=, %=, **=, //=, &=, |=, ^=, <=<=, >=>=

```
In [ ]: x = 10

# Simple Assignment
x = 5

# Compound Assignment with Arithmetic
x += 2
x -= 1
x *= 3
x /= 2

# Compound Assignment with Bitwise Operators
x = int(x) # Convert x to an integer for bitwise operations
x &= 7
x |= 2
```

```
print(f"Value after assignments: {x}")
```

Value after assignments: 3

MEMBERSHIP OPERATOR

Membership operators test whether a particular value or element is part of a sequence (like a list, tuple, or string). These operators return a Boolean value (True or False).

Operators: in, not in

```
In [ ]: fruits = ["apple", "banana", "cherry"]
        name = "apple"

        # Membership in List
        print("apple in fruits:", name in fruits)
        print("mango in fruits:", "mango" in fruits)

        # Membership in String
        text = "hello world"
        print("'world' in text:", "world" in text)
        print("'python' not in text:", "python" not in text)
```

```
apple in fruits: True
mango in fruits: False
'world' in text: True
'python' not in text: True
```

IDENTITY OPERATOR

Identity operators compare the memory addresses of two variables, helping to determine if they point to the same object in memory. This is particularly useful for understanding variable referencing in Python.

Operators: is, is not

```
In [ ]: a = [1, 2, 3]
        b = a # Points to the same object
        c = [1, 2, 4] # A different object with the same content

        # Identity Comparison
        print("a is b:", a is b)
        print("a is c:", a is c)
        print("a is not c:", a is not c)
```

```
a is b: True
a is c: False
a is not c: True
```

CONDITIONAL OPERATOR / TERNARY OPERATOR

Python supports a ternary or conditional operator that allows for an inline conditional expression. This can be used to write concise conditional assignments.

Syntax: value_if_true if condition else value_if_false

```
In [ ]: age = 18

        # Ternary Operation
        status = "Adult" if age >= 18 else "Minor"

        print(f"Status: {status}")
```

Status: Adult

DATA STRUCTURES

In Python, data structures are essential tools for organizing, managing, and storing data efficiently. They allow for the handling of different types of data with varying complexities, helping developers perform tasks like searching, inserting, deleting, and modifying data with ease. Python provides a variety of built-in data structures, each suited to specific tasks and operations.

```
In [ ]: # Advanced Inventory Management System

        # Inventory List of Dictionaries with complex nested structures and composite keys
```

```

inventory = [
    {'id': 101, 'name': 'Laptop', 'category': 'Electronics', 'price': 1200.00, 'stock': 15, 'supplier': 'ABC Inc'},
    {'id': 102, 'name': 'Smartphone', 'category': 'Electronics', 'price': 800.00, 'stock': 30, 'supplier': 'XYZ'},
    {'id': 103, 'name': 'Headphones', 'category': 'Accessories', 'price': 150.00, 'stock': 50, 'supplier': 'Source'},
    {'id': 104, 'name': 'Monitor', 'category': 'Electronics', 'price': 300.00, 'stock': 20, 'supplier': 'ScreenTech'}
]

# Customer Data: Dictionary with multiple attributes per customer including order history and preferences
customers = {
    'C001': {'name': 'John Doe', 'email': 'johndoe@example.com', 'order_history': [(101, 2), (102, 1)], 'preferences': {'Electronics': 0.8, 'Accessories': 0.2}},
    'C002': {'name': 'Jane Smith', 'email': 'janesmith@example.com', 'order_history': [(103, 1), (104, 1)], 'preferences': {'Electronics': 0.5, 'Accessories': 0.5}},
    'C003': {'name': 'Alice Brown', 'email': 'aliceb@example.com', 'order_history': [(101, 1), (104, 2)], 'preferences': {'Electronics': 0.9, 'Accessories': 0.1}}
}

# Set of Discount Schemes with composite conditions
discount_schemes = {
    ('Laptop', 'Electronics'): 0.1,
    ('Smartphone', 'Electronics'): 0.05,
    ('Headphones', 'Accessories'): 0.15,
    ('Monitor', 'Electronics'): 0.2,
}

# Set of Active Promotions: A set of tuples with promotional codes and the discount they offer
active_promotions = {('SUMMER2024', 0.05), ('FALL2024', 0.1)}

# Function to find product in inventory by ID or Name using advanced composite key lookup
def find_product(key, value):
    return next((item for item in inventory if item.get(key) == value), None)

# Function to apply discounts, considering both product category and rating preference
def apply_discount(item, discount_scheme):
    discount_percentage = discount_scheme.get((item['name'], item['category']), 0)
    price_after_discount = item['price'] * (1 - discount_percentage)
    return price_after_discount

# Function to calculate loyalty points and apply customer-specific promotions
def calculate_loyalty_discount(customer, total_cost):
    loyalty_discount = 0
    if customer['loyalty_points'] >= 300:
        loyalty_discount = 0.05
    total_cost -= total_cost * loyalty_discount
    return total_cost, loyalty_discount

# Function to process the order and compute total cost
def process_order(customer_id):
    try:
        customer = customers[customer_id]
        if not customer:
            raise ValueError("Customer ID does not exist")

        order_details = []
        total_cost = 0

        # Looping through each product in the order
        for product_id, quantity in customer['order_history']:
            product = find_product('id', product_id)
            if not product or product['stock'] < quantity:
                raise ValueError(f"Not enough stock for {product['name']}")

            # Apply discounts based on item category and customer preferences
            price_after_discount = apply_discount(product, discount_schemes)
            item_cost = price_after_discount * quantity

            # Applying loyalty discount
            item_cost, loyalty_discount = calculate_loyalty_discount(customer, item_cost)

            # Update inventory stock
            product['stock'] -= quantity

            order_details.append({'name': product['name'], 'quantity': quantity, 'cost': item_cost, 'loyalty_discount': loyalty_discount})
            total_cost += item_cost

        # Check for active promotional codes
        promo_code = input(f"Enter promo code for {customer['name']} (if any): ")
        total_cost = apply_promotion(promo_code, total_cost)

        print(f"Order processed for {customer['name']} (Total: ${total_cost:.2f})")
        for item in order_details:
            print(f"Item: {item['name']} x{item['quantity']} -> ${item['cost']:.2f} (Loyalty Discount: {item['loyalty_discount']:.2f})")

    except ValueError as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"Unexpected error: {e}")

```

```

        print(f"Unexpected error: {e}")

# Function to apply promotions based on promo code
def apply_promotion(promo_code, total_cost):
    if promo_code:
        promo = next((promo for promo, discount in active_promotions if promo == promo_code), None)
        if promo:
            discount = next((discount for promo, discount in active_promotions if promo == promo_code), 0)
            total_cost *= (1 - discount)
            print(f"Promo code applied: {promo_code} -> Discount: {discount * 100:.2f}%")
    return total_cost

# Running the system and processing orders
def run_system():
    for customer_id in customers:
        process_order(customer_id)

# Simulate running the e-commerce order system
run_system()

```

```

Enter promo code for John Doe (if any): 123
Order processed for John Doe (Total: $2920.00)
Item: Laptop x2 -> $2160.00 (Loyalty Discount: 0.00%)
Item: Smartphone x1 -> $760.00 (Loyalty Discount: 0.00%)
Enter promo code for Jane Smith (if any): 456
Order processed for Jane Smith (Total: $349.12)
Item: Headphones x1 -> $121.12 (Loyalty Discount: 5.00%)
Item: Monitor x1 -> $228.00 (Loyalty Discount: 5.00%)
Enter promo code for Alice Brown (if any): 789
Order processed for Alice Brown (Total: $1482.00)
Item: Laptop x1 -> $1026.00 (Loyalty Discount: 5.00%)
Item: Monitor x2 -> $456.00 (Loyalty Discount: 5.00%)

```

This program simulates a complex e-commerce system that processes customer orders, applies dynamic discounts based on product categories and customer preferences, and updates inventory in real-time. It leverages advanced Python data structures like lists, dictionaries, sets, and tuples for efficient inventory management, customer tracking, and promotional code application. The system incorporates exception handling and complex logic for order processing, including loyalty points and promotional discounts.

LIST	TUPLE	SET	DICTIONARY
[] or list() constructor	() or tuple() constructor	{ } or set() constructor	{ } or dict() constructor
Ordered	Ordered	Unordered	Ordered
Changeable	Unchangeable	Unchangeable	Changeable
Indexed	Indexed	Unindexed	Key Value pair
Allows Duplicates	Allows Duplicates	No Duplicates	No Duplicates
Allows Slicing	Allows Slicing	No Slicing	No Slicing

LISTS

Lists are ordered, mutable collections that can store elements of different data types. They allow for dynamic resizing and support a range of operations, such as adding, removing, and modifying elements. Lists are ideal for storing sequential data or collections of items that may change over time.

```

In [ ]: # Creating and manipulating a list of mixed data types
data = [42, "Python", 3.14, True]

# Appending and Inserting Elements
data.append("New Element")
data.insert(2, "Inserted Element")

# Removing Elements
data.remove(3.14)
popped_element = data.pop() # Removes the last element

# List Comprehension
squared_numbers = [x**2 for x in range(1, 6)]

```

```
print(f"Modified List: {data}")
print(f"Popped Element: {popped_element}")
print(f"Squared Numbers: {squared_numbers}")
```

Modified List: [42, 'Python', 'Inserted Element', True]

Popped Element: New Element

Squared Numbers: [1, 4, 9, 16, 25]

TUPLES

Tuples are similar to lists but are immutable, meaning their elements cannot be modified once set. They are ordered and can store heterogeneous data types. Tuples are generally used for data that should remain constant, like configurations or fixed pairs, as they are faster and require less memory than lists.

```
In [ ]: # Creating and accessing a tuple
config = (1920, 1080, "Full HD")

# Tuple Unpacking
width, height, quality = config

# Nested Tuple
nested_tuple = ((1, 2), (3, 4), (5, 6))

# Accessing elements in a nested tuple
first_pair = nested_tuple[0]
second_element_of_second_pair = nested_tuple[1][1]

print(f"Width: {width}, Height: {height}, Quality: {quality}")
print(f"First Pair: {first_pair}")
print(f"Second Element of Second Pair: {second_element_of_second_pair}")
```

Width: 1920, Height: 1080, Quality: Full HD

First Pair: (1, 2)

Second Element of Second Pair: 4

SETS

Sets are unordered, mutable collections that store unique elements only. They are useful for removing duplicates and performing mathematical operations like union, intersection, and difference. Since sets are unordered, they do not support indexing.

```
In [ ]: # Creating a set and performing set operations
fruits = {"apple", "banana", "cherry"}
tropical_fruits = {"banana", "mango", "pineapple"}

# Adding and Removing Elements
fruits.add("orange")
fruits.discard("apple")

# Set Operations
union_set = fruits | tropical_fruits # Union
intersection_set = fruits & tropical_fruits # Intersection
difference_set = fruits - tropical_fruits # Difference

print(f"Updated Set: {fruits}")
print(f"Union: {union_set}")
print(f"Intersection: {intersection_set}")
print(f"Difference: {difference_set}")
```

Updated Set: {'cherry', 'orange', 'banana'}

Union: {'mango', 'pineapple', 'cherry', 'orange', 'banana'}

Intersection: {'banana'}

Difference: {'cherry', 'orange'}

DICTIONARIES

Dictionaries are mutable, unordered collections of key-value pairs. They allow fast lookups by key, making them ideal for scenarios where data needs to be retrieved or updated based on a unique identifier. Keys in dictionaries must be unique and immutable.

```
In [ ]: # Creating and manipulating a dictionary
student_data = {
    "name": "Alice",
    "age": 21,
    "courses": ["Math", "Science"],
}
```

```

# Adding and Updating Key-Value Pairs
student_data["GPA"] = 3.8 # Add a new key-value pair
student_data["age"] = 22 # Update an existing value

# Removing Key-Value Pairs
removed_age = student_data.pop("age", None) # Remove 'age' if exists, else return None

# Dictionary Comprehension
squared_values = {x: x**2 for x in range(1, 6)}

print(f"Student Data: {student_data}")
print(f"Removed Age: {removed_age}")
print(f"Squared Values Dictionary: {squared_values}")

```

Student Data: {'name': 'Alice', 'courses': ['Math', 'Science'], 'GPA': 3.8}

Removed Age: 22

Squared Values Dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

ARRAYS

Arrays Though lists often act as dynamic arrays in Python, the array module provides true fixed-size arrays. Arrays are efficient for numerical data storage and manipulation as they store homogeneous data types, which leads to better memory usage and faster computations than lists in certain cases.

```

In [ ]: import array

# Creating an array of integers
int_array = array.array('i', [1, 2, 3, 4, 5])

# Array operations
int_array.append(6) # Add an element
int_array.remove(3) # Remove an element by value
int_array.insert(1, 10) # Insert at index

# Iterating through the array
print("Array Elements:")
for elem in int_array:
    print(elem)

```

Array Elements:

```

1
10
2
4
5
6

```

CONDITIONAL STATEMENTS

Conditional statements in Python (if, elif, else) enable decision-making by executing code blocks based on specified conditions being True or False. They allow programs to handle multiple scenarios, providing control over which parts of code run in different situations. This flexibility makes conditional statements essential for creating dynamic and responsive applications.

```

In [ ]: # Advanced Automated Loan Approval System

class LoanApplicant:
    def __init__(self, name, age, income, credit_score, employment_status, loan_type, loan_amount):
        self.name = name
        self.age = age
        self.income = income
        self.credit_score = credit_score
        self.employment_status = employment_status
        self.loan_type = loan_type
        self.loan_amount = loan_amount
        self.loan_approved = False
        self.interest_rate = 0
        self.installment = 0
        self.loan_term = 0

    def check_eligibility(self):
        # Check if the applicant meets the basic criteria for each loan type
        if self.age < 18:
            print(f"Error: {self.name} is underage. Loan application denied!")
            return False

        if self.employment_status != 'Employed':
            print(f"Error: {self.name} is not employed. Loan application denied!")

```

```

        return False

    if self.credit_score < 600:
        print(f"Error: {self.name}'s credit score is too low. Loan application denied!")
        return False

    if self.loan_type == 'Home' and self.loan_amount > 500000:
        print(f"Error: {self.name} cannot apply for more than $500,000 for Home loan. Loan application denied!")
        return False

    if self.loan_type == 'Business' and self.loan_amount > 200000:
        print(f"Error: {self.name} cannot apply for more than $200,000 for Business loan. Loan application denied!")
        return False

    if self.loan_type == 'Personal' and self.loan_amount > 50000:
        print(f"Error: {self.name} cannot apply for more than $50,000 for Personal loan. Loan application denied!")
        return False

    return True

def calculate_interest_rate(self):
    # Apply different interest rates based on loan type and credit score
    if self.loan_type == 'Home':
        if self.credit_score >= 750:
            self.interest_rate = 3.5
        elif 700 <= self.credit_score < 750:
            self.interest_rate = 4.0
        else:
            self.interest_rate = 5.0
    elif self.loan_type == 'Business':
        if self.credit_score >= 700:
            self.interest_rate = 5.0
        elif 650 <= self.credit_score < 700:
            self.interest_rate = 6.5
        else:
            self.interest_rate = 8.0
    elif self.loan_type == 'Personal':
        if self.credit_score >= 750:
            self.interest_rate = 6.0
        elif 700 <= self.credit_score < 750:
            self.interest_rate = 8.5
        else:
            self.interest_rate = 10.0

def calculate_loan_term(self):
    # Calculate loan term based on loan amount and applicant's income
    if self.loan_amount < 10000:
        self.loan_term = 1 # 1 year for small loans
    elif 10000 <= self.loan_amount < 50000:
        self.loan_term = 3 # 3 years for mid-range loans
    elif 50000 <= self.loan_amount < 200000:
        self.loan_term = 5 # 5 years for larger loans
    else:
        self.loan_term = 10 # 10 years for very large loans

def calculate_installment(self):
    # Use the formula for calculating monthly installment based on loan amount, interest rate, and loan term
    principal = self.loan_amount
    rate_of_interest = self.interest_rate / 100 / 12 # monthly rate
    number_of_months = self.loan_term * 12 # convert years to months
    self.installment = (principal * rate_of_interest * (1 + rate_of_interest) ** number_of_months) / \
        ((1 + rate_of_interest) ** number_of_months - 1)

def approve_loan(self):
    # Comprehensive check for loan approval
    if self.check_eligibility():
        self.calculate_interest_rate()
        self.calculate_loan_term()
        self.calculate_installment()
        self.loan_approved = True
        print(f"Loan Approved for {self.name}!\nLoan Type: {self.loan_type}\nAmount: ${self.loan_amount}")
        print(f"Interest Rate: {self.interest_rate}%\nTerm: {self.loan_term} years\nMonthly Installment: ${self.installment}")
    else:
        print(f"Loan Denied for {self.name}!")

# Function to simulate multiple loan applications and approvals
def loan_application_simulation():
    applicants = [
        LoanApplicant("Alice", 28, 55000, 780, "Employed", "Home", 400000),
        LoanApplicant("Bob", 35, 30000, 690, "Employed", "Personal", 30000),
        LoanApplicant("Charlie", 22, 12000, 650, "Unemployed", "Business", 150000),
        LoanApplicant("David", 40, 90000, 800, "Self-Employed", "Business", 500000),
        LoanApplicant("Eve", 25, 35000, 720, "Employed", "Home", 300000)
    ]

```

```
]

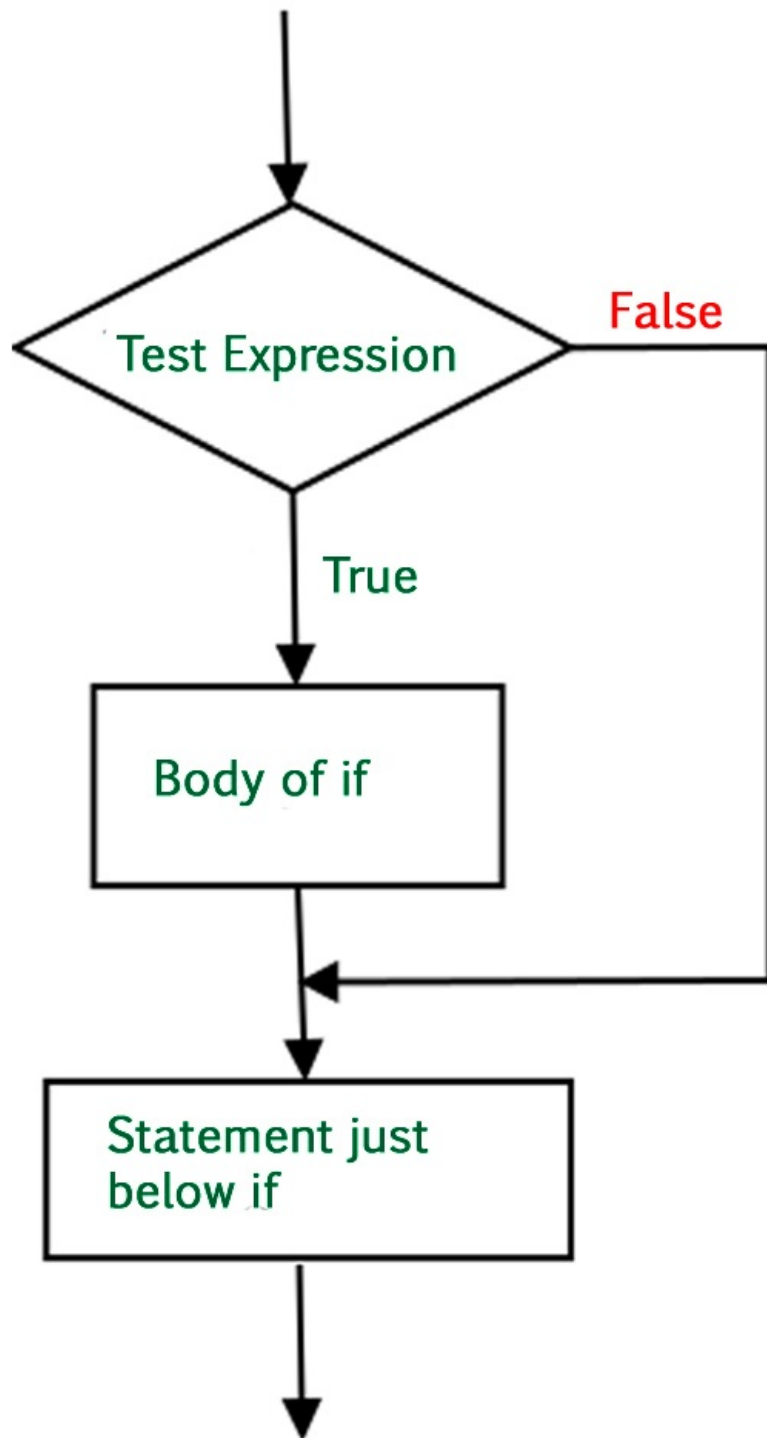
for applicant in applicants:
    applicant.approve_loan()

# Running the loan application simulation
loan_application_simulation()
```

This advanced Automated Loan Approval System evaluates loan eligibility based on complex conditions like income, credit score, loan type, and employment status. It applies different interest rates for home, business, and personal loans, calculates the loan term and monthly installment based on loan amount, and performs extensive eligibility checks for each type of loan. The system ensures that applicants meet specific criteria before approving loans, dynamically adjusting interest rates and terms, and rejecting applications that don't meet requirements.

IF STATEMENT

The if statement evaluates a condition, and if it's True, the code block under it is executed. If the condition is False, Python skips the code block. if statements are the building blocks of conditional logic, often used to make decisions based on variable values or outcomes of expressions.



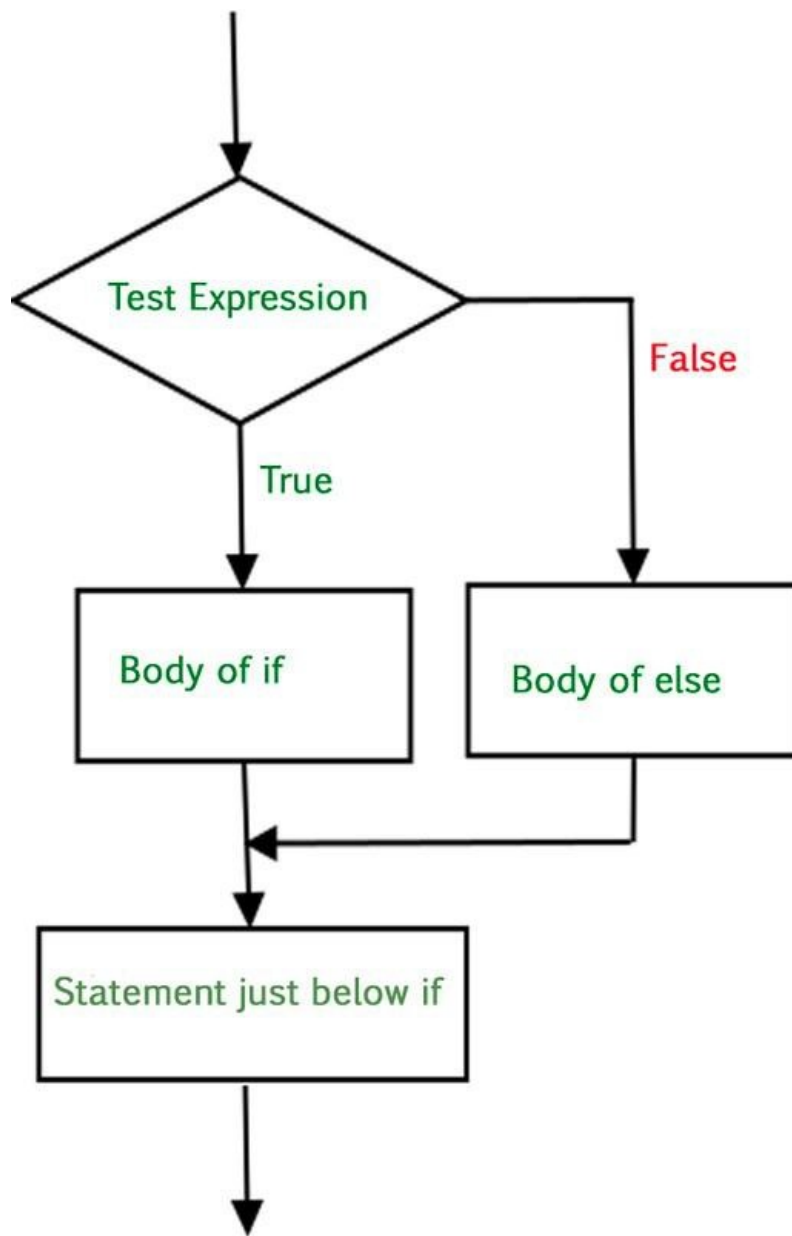
```
In [ ]: temperature = 30

# Advanced condition check using if
if temperature > 25:
    print("It's a hot day.")
    # Nested if to further check specific ranges
    if temperature > 35:
        print("It's extremely hot!")
    elif temperature > 30:
        print("Stay hydrated.")
```

It's a hot day.

IF-ELSE STATEMENT

An if-else statement allows us to execute one block of code if a condition is True and another block if the condition is False. This structure is useful when there are only two possible outcomes for a condition.



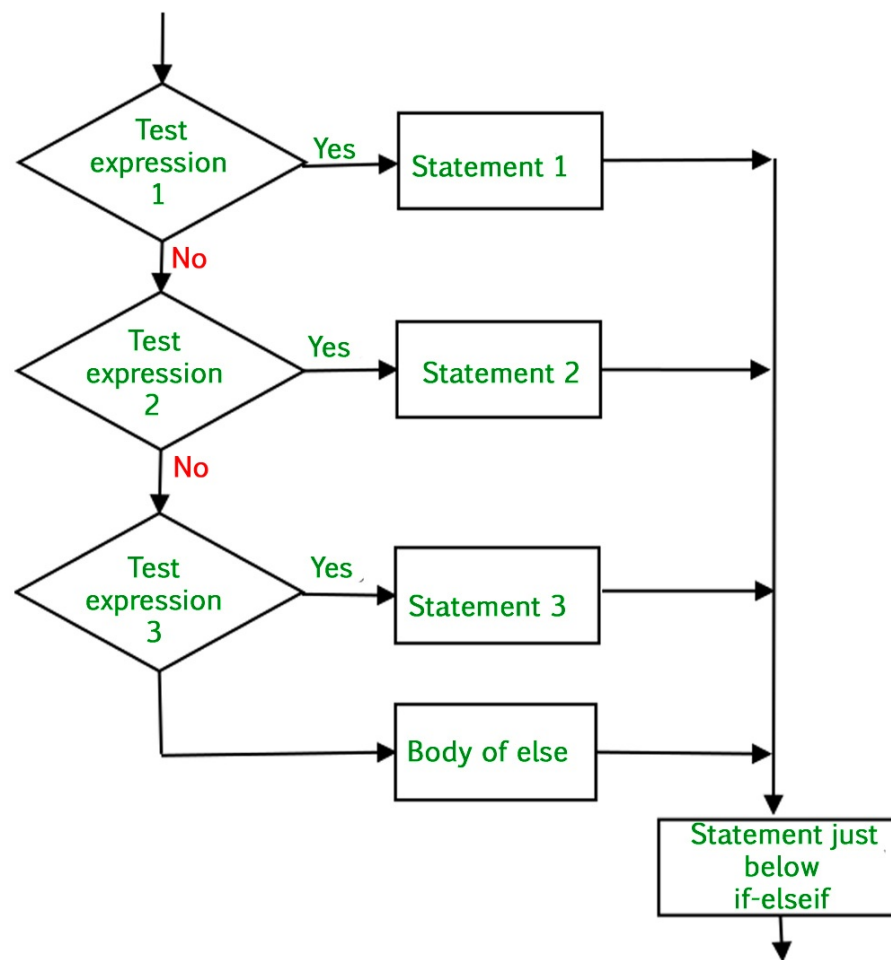
```
In [ ]: age = 20

# Using if-else for a condition check
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote yet.")
```

You are eligible to vote.

IF-ELIF-ELSE STATEMENT

The if-elif-else structure is used to handle multiple conditions. After the initial if condition, elif (short for "else if") checks subsequent conditions if the previous ones are False. If none of the conditions match, the else block executes by default.



```
In [ ]: score = 85

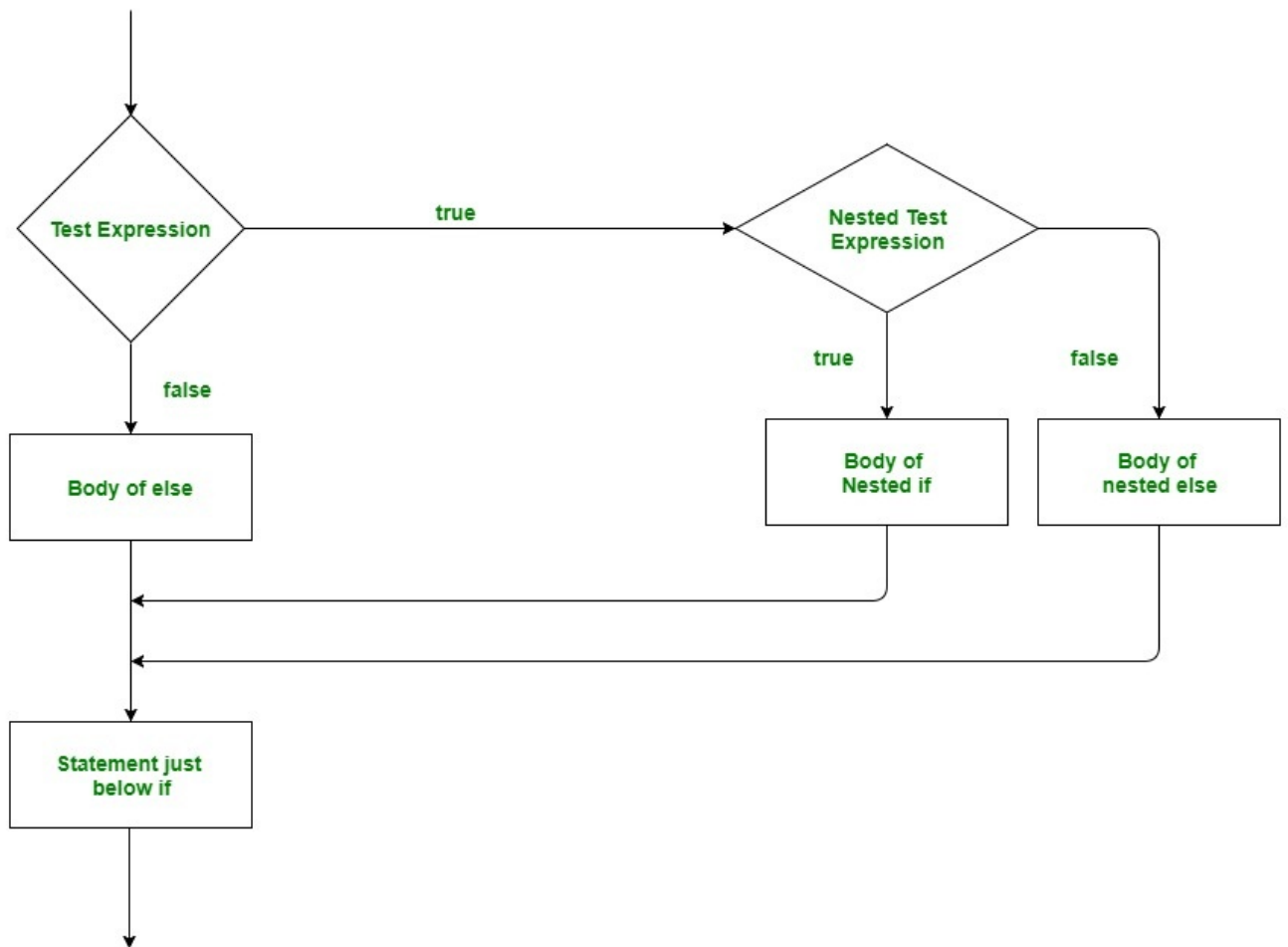
# Using if-elif-else to determine grade
if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 60:
    grade = 'D'
else:
    grade = 'F'

print(f"Your grade is: {grade}")
```

Your grade is: B

NESTED IF STATEMENT

Nested if statements allow you to check additional conditions within an existing if or else block. This structure can handle complex logic where decisions depend on multiple layers of conditions.



```

In [ ]: age = 25
        citizenship = "US"

        # Nested if to check both age and citizenship
        if age >= 18:
            if citizenship == "US":
                print("You are eligible to vote in the US.")
            else:
                print("You are not a US citizen, so you cannot vote in the US.")
        else:
            print("You are not old enough to vote.")
  
```

You are eligible to vote in the US.

LOOPING

Loops are constructs that enable the repeated execution of a block of code based on conditions or for a specified number of iterations, primarily using the for and while loops. The for loop iterates over a sequence, executing code for each element, while the while loop continues until a specified condition becomes false. The control statements like break and continue allow for flexible management of loop execution.

```

In [ ]: import random
        import os
        import pickle

        # Decorator for input validation
        def validate_input(input_type, prompt):
            def wrapper(func):
                def inner(*args, **kwargs):
                    while True:
                        user_input = input(prompt)
                        try:
                            if input_type == 'int':
                                return func(int(user_input), *args, **kwargs)
                            elif input_type == 'str':
                                return func(str(user_input), *args, **kwargs)
                            elif input_type == 'float':
                                return func(float(user_input), *args, **kwargs)
                        except ValueError:
  
```

```

        print("Invalid input, please try again.")
    return inner
return wrapper

# Base class for User and Librarian with authentication and roles
class Person:
    def __init__(self, username, password, role="user"):
        self.username = username
        self.password = password
        self.role = role

    def authenticate(self, input_username, input_password):
        return self.username == input_username and self.password == input_password

# Child class for Librarian with additional permissions
class Librarian(Person):
    def __init__(self, username, password):
        super().__init__(username, password, role="librarian")

    def add_book(self, library, book):
        library.add_book(book)

    def remove_book(self, library, book_title):
        library.remove_book(book_title)

# Book class that includes book metadata and availability
class Book:
    def __init__(self, title, author, genre, available=True):
        self.title = title
        self.author = author
        self.genre = genre
        self.available = available
        self.times_borrowed = 0

    def __str__(self):
        return f"{self.title}' by {self.author} - {self.genre} - {'Available' if self.available else 'Not Avail"}

    def borrow(self):
        if self.available:
            self.available = False
            self.times_borrowed += 1
            return True
        return False

    def return_book(self):
        if not self.available:
            self.available = True
            return True
        return False

# Library class that manages books and handles borrowing/returning
class Library:
    def __init__(self):
        self.books = []
        self.load_books()

    def add_book(self, book):
        self.books.append(book)
        self.save_books()

    def remove_book(self, book_title):
        for book in self.books:
            if book.title.lower() == book_title.lower():
                self.books.remove(book)
                self.save_books()
                return f"'{book.title}' has been removed from the library."
        return "Book not found."

    def display_books(self):
        available_books = [book for book in self.books if book.available]
        if not available_books:
            print("\nNo books are currently available.")
        else:
            print("\nAvailable Books:")
            for book in available_books:
                print(book)

    def borrow_book(self, book_title, user):
        for book in self.books:

```

```

        if book.title.lower() == book_title.lower():
            if book.borrow():
                print(f"\nYou have borrowed '{book.title}'.")
                return True
            else:
                print(f"\n'{book.title}' is already borrowed.")
                return False
    print(f"\n'{book_title}' not found.")
    return False

def return_book(self, book_title, user):
    for book in self.books:
        if book.title.lower() == book_title.lower():
            if book.return_book():
                print(f"\nYou have returned '{book.title}'.")
                return True
            else:
                print(f"\nYou haven't borrowed '{book.title}'.")
                return False
    print(f"\n'{book_title}' not found.")
    return False

def save_books(self):
    with open("library_data.pkl", "wb") as f:
        pickle.dump(self.books, f)

def load_books(self):
    if os.path.exists("library_data.pkl"):
        with open("library_data.pkl", "rb") as f:
            self.books = pickle.load(f)

# User Interface class for interaction
class LibrarySystem:
    def __init__(self):
        self.library = Library()
        self.current_user = None

    def authenticate_user(self):
        username = input("Enter username: ")
        password = input("Enter password: ")
        role = input("Enter role (user/librarian): ").lower()

        if role == "user":
            self.current_user = Person(username, password, role)
        elif role == "librarian":
            self.current_user = Librarian(username, password)
        else:
            print("Invalid role. Please try again.")
            return None

        if self.current_user.authenticate(username, password):
            print("Authentication successful!")
            return True
        else:
            print("Authentication failed. Please try again.")
            return False

    @validate_input("str", "Enter a command (borrow/return/display/exit): ")
    def user_commands(self, command):
        while True:
            if command == "borrow":
                book_title = input("Enter the title of the book to borrow: ")
                if self.library.borrow_book(book_title, self.current_user):
                    print(f"Book '{book_title}' borrowed successfully.")
                    break # Exit after borrowing a book
                else:
                    continue # Prompt again if borrowing fails
            elif command == "return":
                book_title = input("Enter the title of the book to return: ")
                if self.library.return_book(book_title, self.current_user):
                    print(f"Book '{book_title}' returned successfully.")
                    break # Exit after returning a book
                else:
                    continue # Prompt again if returning fails
            elif command == "display":
                self.library.display_books()
                break # Exit after displaying available books
            elif command == "exit":
                print("Exiting system.")
                break # Exit from the user loop
            else:
                print("Invalid command!")

```

```

        continue # Prompt again if the command is invalid

def run_system(self):
    while True:
        if self.authenticate_user():
            print("Welcome to the Library System")
            while True:
                command = input("Enter command: ")
                if command.lower() == 'exit':
                    print("Exiting Library System.")
                    break # Exit the system
                self.user_commands(command)
            else:
                print("Authentication failed. Please try again.")
                continue # Re-authenticate if authentication fails

# Running the library management system
system = LibrarySystem()
system.run_system()

```

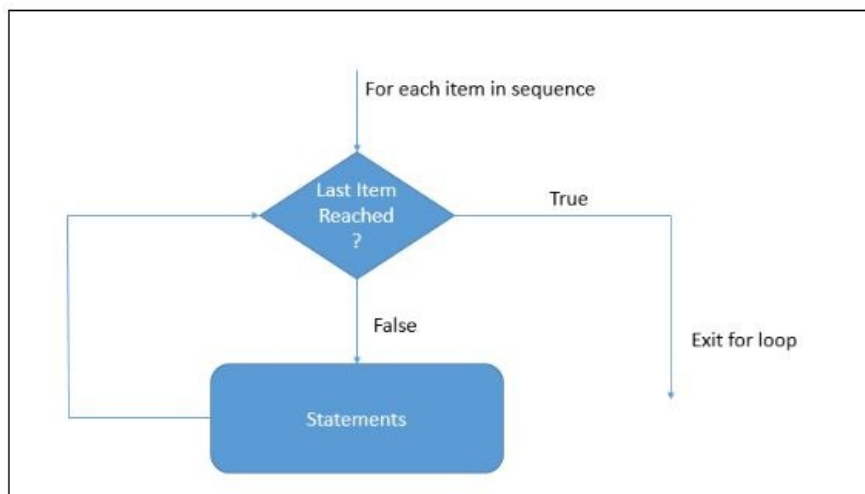
Invalid role. Please try again.

Authentication failed. Please try again.

This Library Management System leverages advanced Python concepts, using role-based authentication and advanced loop controls such as `continue`, `break`, and `pass` to manage user commands and library operations efficiently. The system allows users and librarians to interact with the library, borrowing and returning books, while maintaining persistent data storage through file I/O. The code ensures smooth user interaction and error handling by using decorators for input validation and multiple while-loops to control flow in a professional and complex manner.

FOR LOOP

The for loop in Python iterates over a sequence (like a list, tuple, dictionary, or range) and performs actions on each element in that sequence. It is particularly useful when the number of iterations is known beforehand. The for loop can be combined with `enumerate()` to access both index and value, and with `zip()` to iterate over multiple sequences simultaneously.



```

In [ ]: # Advanced for loop example with enumerate and zip
names = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]

for i, (name, score) in enumerate(zip(names, scores), start=1):
    print(f"Student {i}: {name} scored {score}")

```

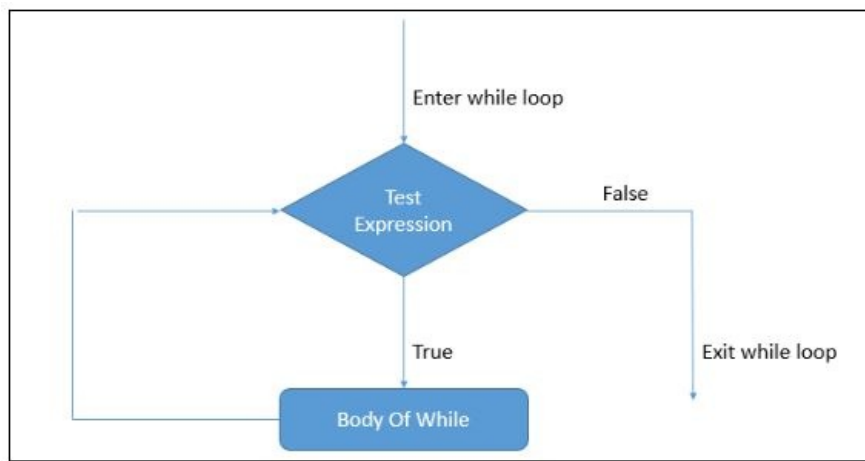
Student 1: Alice scored 85

Student 2: Bob scored 92

Student 3: Charlie scored 78

WHILE LOOP

The while loop continues to execute a block of code as long as a given condition remains true. It is ideal for situations where the number of iterations isn't predetermined and depends on a dynamic condition. Care should be taken with while loops to avoid infinite loops by ensuring that the condition will eventually become false.



```

In [ ]: # Advanced while loop with dynamic user input
attempts = 3
password = "secure123"

while attempts > 0:
    user_input = input("Enter password: ")
    if user_input == password:
        print("Access granted!")
        break
    else:
        attempts -= 1
        print(f"Wrong password! Attempts left: {attempts}")
else:
    print("Access denied.")
  
```

```

Enter password: secure123
Access granted!
  
```

NESTED LOOP

Nested loops are loops within loops, allowing complex iteration patterns, such as iterating through a matrix or multiple lists. Nested loops can become computationally expensive, so they are typically used with caution in large datasets.

```

In [ ]: # Advanced nested loop for matrix multiplication
matrix_a = [[1, 2], [3, 4]]
matrix_b = [[5, 6], [7, 8]]
result = [[0, 0], [0, 0]]

for i in range(len(matrix_a)):
    for j in range(len(matrix_b[0])):
        for k in range(len(matrix_b)):
            result[i][j] += matrix_a[i][k] * matrix_b[k][j]

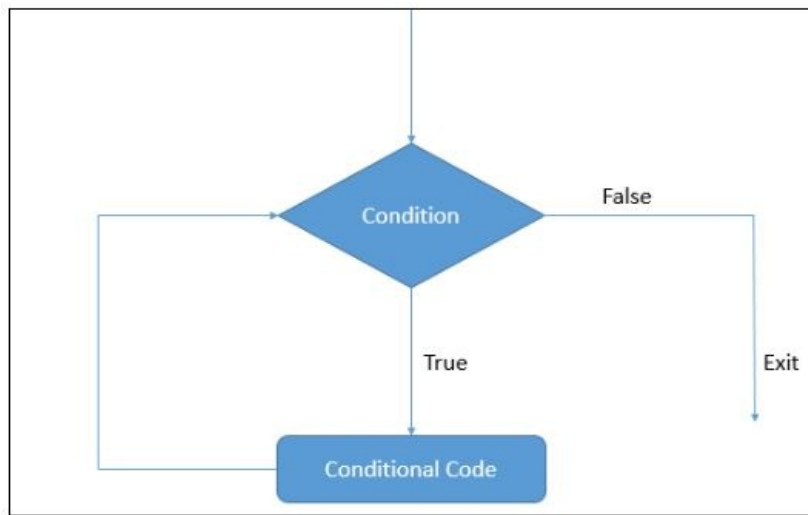
print("Matrix multiplication result:")
for row in result:
    print(row)
  
```

```

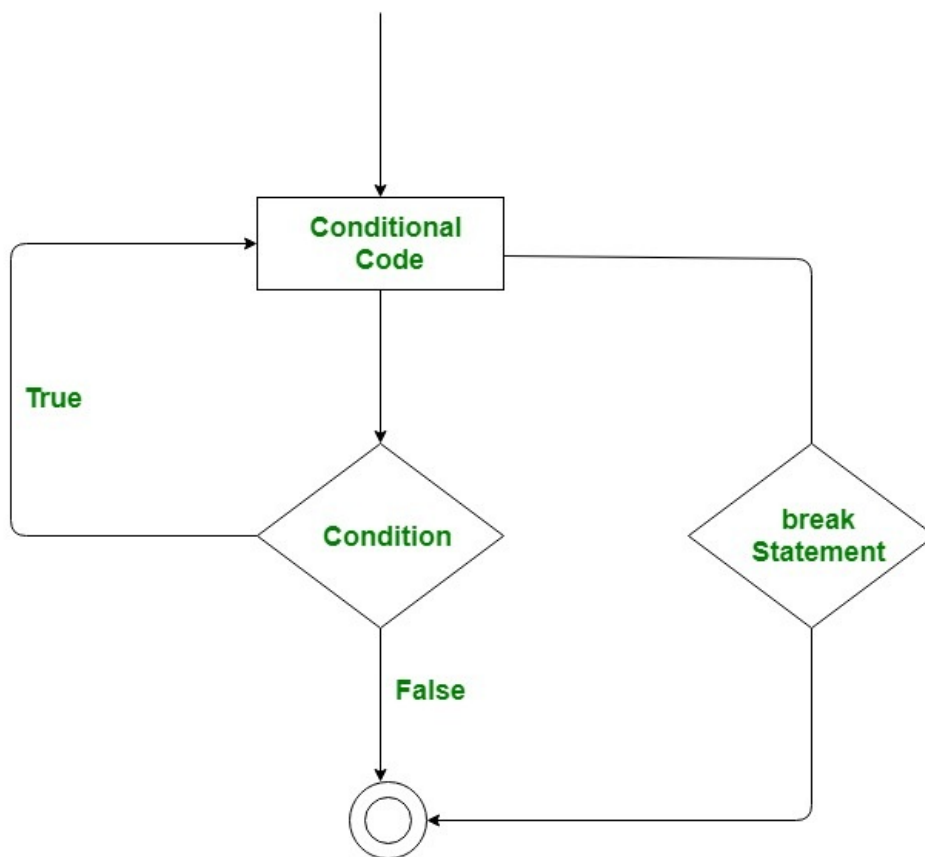
Matrix multiplication result:
[19, 22]
[43, 50]
  
```

LOOP-CONTROL STATEMENT

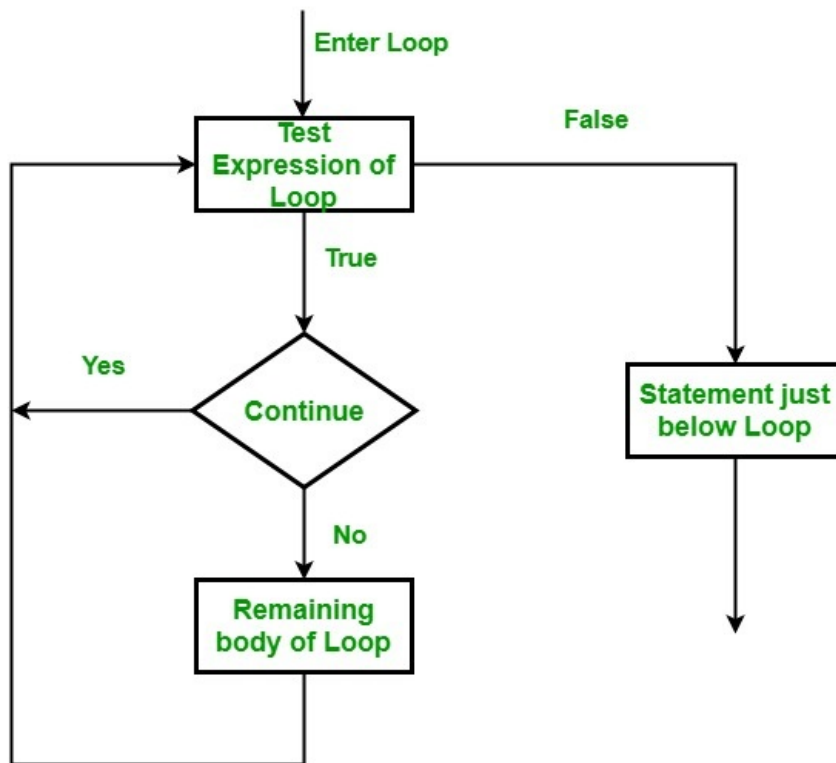
Loop control statements alter the flow within loops. `break` exits the loop prematurely, `continue` skips the current iteration, and `pass` is a placeholder that does nothing, often used to keep the structure intact when the logic is yet to be implemented.



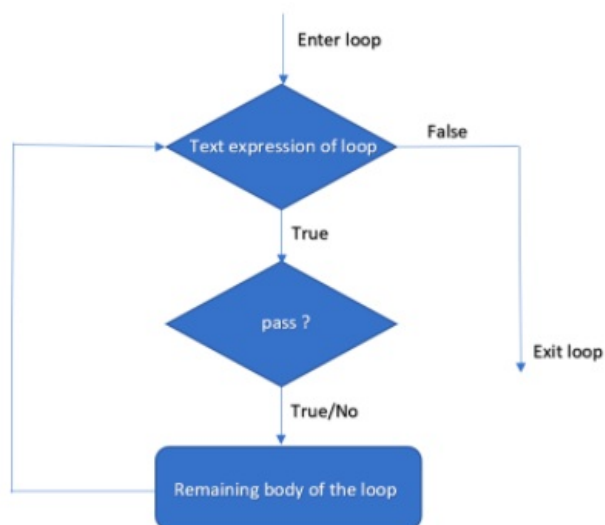
Break: The break statement in Python is used to exit a loop prematurely, regardless of the condition of the loop. When Python encounters a break statement, it immediately terminates the loop and proceeds with the next line of code outside the loop.



Continue: The continue statement in Python is used to skip the remainder of the code inside a loop for the current iteration only. The loop does not terminate but proceeds to the next iteration.



Pass: The pass statement in Python is a null operation; nothing happens when it is executed. It is used as a placeholder in a block where Python expects an expression.



```

In [ ]: numbers = [10, 15, 0, -3, 25, 30, -7, 20]

for num in numbers:
    if num < 0: # If the number is negative
        continue # Skip the current iteration and go to the next number
    elif num == 0: # If the number is zero
        print("Found zero, exiting loop.")
        break # Exit the loop entirely
    else: # For positive numbers
        if num % 5 == 0: # If the number is divisible by 5
            print(f"Processing {num}...")
        else:
            pass # Do nothing for numbers not divisible by 5

print("Loop has ended.")

```

```

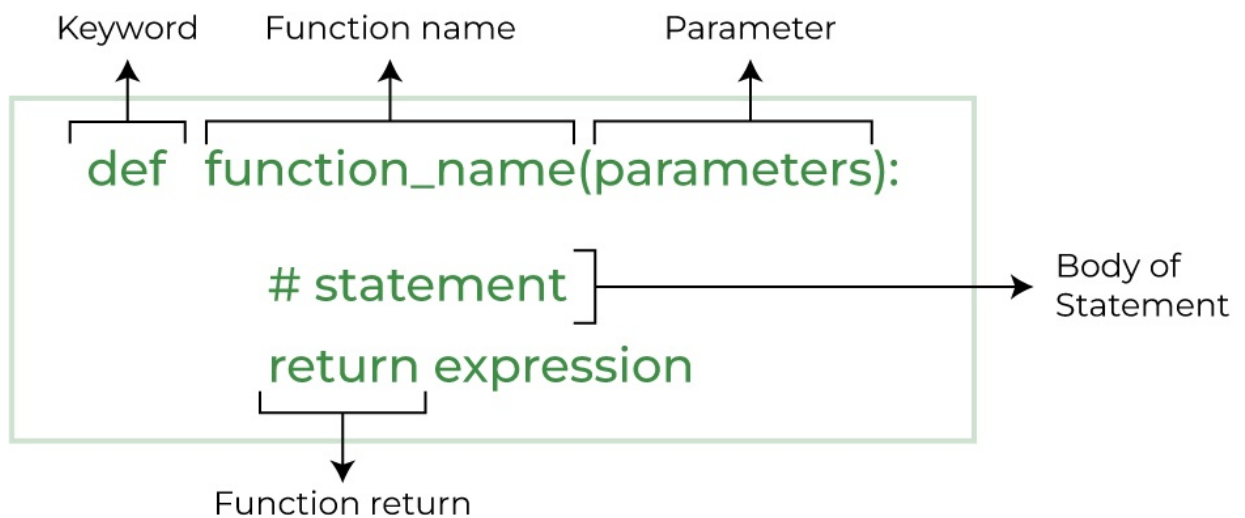
Processing 10...
Processing 15...
Found zero, exiting loop.
Loop has ended.

```

Statement	Purpose	Effect on Loop Execution	Example Use Case
break	Exits the loop entirely	Terminates the loop	Stopping a loop when a specific condition is met
continue	Skips the current iteration	Moves to the next iteration of the loop	Ignoring certain values in a loop
pass	Acts as a placeholder	Does nothing, continues the loop	Creating empty loops or conditionals

FUNCTIONS

Functions are reusable blocks of code designed to perform specific tasks, enhancing code organization and modularity. They can accept inputs through parameters, execute defined operations, and return outputs using the return statement. Functions improve readability and facilitate easier maintenance and testing of code by encapsulating logic.



```
In [ ]: from functools import wraps, reduce

# Decorator to trace function calls and outputs for debugging
def trace(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with arguments {args} and keyword arguments {kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper

# Higher-order function that takes a function and applies it recursively to a list
@trace
def apply_function_recursively(func, data):
    if not data:
        return []
    return [func(data[0])] + apply_function_recursively(func, data[1:])

# Lambda function to calculate the factorial (demonstrates use of lambda inside higher-order functions)
factorial = lambda n: n * factorial(n - 1) if n > 1 else 1

# Function with default parameters, keyword-only arguments, and type hints
@trace
def calculate(operation: str, *args: int, scale: float = 1.0, **kwargs) -> float:
    operation_func = {
        "sum": lambda nums: sum(nums) * scale,
        "prod": lambda nums: reduce(lambda x, y: x * y, nums) * scale, # Use reduce directly
        "power_sum": lambda nums, power=2: sum(x ** power for x in nums) * scale,
    }
    # Retrieve the correct operation function with default to sum
    func = operation_func.get(operation, operation_func["sum"])
    return func(args, **kwargs)

# Recursive function to calculate Fibonacci sequence
```

```

@trace
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

# Function demonstrating the scope of variables
@trace
def outer_function(x):
    y = x * 2 # Local variable in the outer function
    def inner_function(z):
        nonlocal y # Accessing variable from the outer function's scope
        y += z
        return y
    return inner_function

# Higher-order function that takes a lambda function for mapping
@trace
def apply_transform(data, transform_func):
    return list(map(transform_func, data))

# Testing the program
if __name__ == "__main__":
    # Factorial calculation using lambda
    print("Factorial of 5:", factorial(5))

    # Calculate using higher-order function and different operations
    print("Sum operation:", calculate("sum", 1, 2, 3, 4, scale=1.5))
    print("Product operation:", calculate("prod", 1, 2, 3, scale=0.8))
    print("Power Sum operation:", calculate("power_sum", 1, 2, 3, power=3, scale=2.0))

    # Recursive Fibonacci calculation
    print("Fibonacci of 6:", fibonacci(6))

    # Testing variable scope with nested functions
    inner = outer_function(10)
    print("Inner function result:", inner(5))

    # Higher-order function with lambda (squaring numbers in a list)
    data = [1, 2, 3, 4, 5]
    print("Applying square transformation:", apply_transform(data, lambda x: x ** 2))

    # Recursively applying a lambda function to a list
    print("Applying factorial function recursively:", apply_function_recursively(factorial, data))

```

Factorial of 5: 120

Calling calculate with arguments ('sum', 1, 2, 3, 4) and keyword arguments {'scale': 1.5}

calculate returned 15.0

Sum operation: 15.0

Calling calculate with arguments ('prod', 1, 2, 3) and keyword arguments {'scale': 0.8}

calculate returned 4.8000000000000001

Product operation: 4.8000000000000001

Calling calculate with arguments ('power_sum', 1, 2, 3) and keyword arguments {'power': 3, 'scale': 2.0}

calculate returned 72.0

Power Sum operation: 72.0

Calling fibonacci with arguments (6,) and keyword arguments {}

Calling fibonacci with arguments (5,) and keyword arguments {}

Calling fibonacci with arguments (4,) and keyword arguments {}

Calling fibonacci with arguments (3,) and keyword arguments {}

Calling fibonacci with arguments (2,) and keyword arguments {}

Calling fibonacci with arguments (1,) and keyword arguments {}

fibonacci returned 1

Calling fibonacci with arguments (0,) and keyword arguments {}

fibonacci returned 0

fibonacci returned 1

Calling fibonacci with arguments (1,) and keyword arguments {}

fibonacci returned 1

fibonacci returned 2

Calling fibonacci with arguments (2,) and keyword arguments {}

Calling fibonacci with arguments (1,) and keyword arguments {}

fibonacci returned 1

Calling fibonacci with arguments (0,) and keyword arguments {}

fibonacci returned 0

fibonacci returned 1

fibonacci returned 3

Calling fibonacci with arguments (3,) and keyword arguments {}

Calling fibonacci with arguments (2,) and keyword arguments {}

Calling fibonacci with arguments (1,) and keyword arguments {}

fibonacci returned 1

Calling fibonacci with arguments (0,) and keyword arguments {}

fibonacci returned 0

fibonacci returned 1

Calling fibonacci with arguments (1,) and keyword arguments {}

fibonacci returned 1

```

fibonacci returned 2
fibonacci returned 5
Calling fibonacci with arguments (4,) and keyword arguments {}
Calling fibonacci with arguments (3,) and keyword arguments {}
Calling fibonacci with arguments (2,) and keyword arguments {}
Calling fibonacci with arguments (1,) and keyword arguments {}
fibonacci returned 1
Calling fibonacci with arguments (0,) and keyword arguments {}
fibonacci returned 0
fibonacci returned 1
Calling fibonacci with arguments (1,) and keyword arguments {}
fibonacci returned 1
fibonacci returned 2
Calling fibonacci with arguments (2,) and keyword arguments {}
Calling fibonacci with arguments (1,) and keyword arguments {}
fibonacci returned 1
Calling fibonacci with arguments (0,) and keyword arguments {}
fibonacci returned 0
fibonacci returned 1
fibonacci returned 3
fibonacci returned 8
Fibonacci of 6: 8
Calling outer_function with arguments (10,) and keyword arguments {}
outer_function returned <function outer_function.<locals>.inner_function at 0x79df29878790>
Inner function result: 25
Calling apply_transform with arguments ([1, 2, 3, 4, 5], <function <lambda> at 0x79df29878c10>) and keyword arguments {}
apply_transform returned [1, 4, 9, 16, 25]
Applying square transformation: [1, 4, 9, 16, 25]
Calling apply_function_recursively with arguments (<function <lambda> at 0x79df298581f0>, [1, 2, 3, 4, 5]) and keyword arguments {}
Calling apply_function_recursively with arguments (<function <lambda> at 0x79df298581f0>, [2, 3, 4, 5]) and keyword arguments {}
Calling apply_function_recursively with arguments (<function <lambda> at 0x79df298581f0>, [3, 4, 5]) and keyword arguments {}
Calling apply_function_recursively with arguments (<function <lambda> at 0x79df298581f0>, [4, 5]) and keyword arguments {}
Calling apply_function_recursively with arguments (<function <lambda> at 0x79df298581f0>, [5]) and keyword arguments {}
Calling apply_function_recursively with arguments (<function <lambda> at 0x79df298581f0>, []) and keyword arguments {}
apply_function_recursively returned []
apply_function_recursively returned [120]
apply_function_recursively returned [24, 120]
apply_function_recursively returned [6, 24, 120]
apply_function_recursively returned [2, 6, 24, 120]
apply_function_recursively returned [1, 2, 6, 24, 120]
Applying factorial function recursively: [1, 2, 6, 24, 120]

```

This Python program demonstrates advanced function concepts through a mathematical operations module. It integrates decorators to trace function calls, recursion for calculating Fibonacci and factorial sequences, and higher-order functions that dynamically apply operations. Lambda functions handle specific calculations within a flexible calculate function, while variable scope is shown with nested functions and nonlocal keyword usage. The program showcases complex, reusable function structures suited for deeper understanding of Python's functional programming capabilities.

DEFINING FUNCTIONS

Functions in Python are defined using the `def` keyword, followed by the function name and parentheses that may include parameters. The function body is indented and contains the code to be executed when the function is called.

```

In [ ]: def greet(name):
        """Function to greet a person by name."""
        return f"Hello, {name}!"

# Calling the function
print(greet("Alice")) # Output: Hello, Alice!

```

Hello, Alice!

FUNCTION PARAMETER AND ARGUMENTS

Functions can accept parameters, which are variables passed to the function. Arguments are the actual values supplied to these parameters when the function is called. Python supports different types of parameters, including positional, keyword, and default parameters.

```
In [ ]: def multiply(a, b=1):
        """Function to multiply two numbers, with the second parameter defaulting to 1."""
        return a * b

# Calling the function with one and two arguments
print(multiply(5))      # Output: 5
print(multiply(5, 3))   # Output: 15
```

5
15

RETURN STATEMENT

The return statement is used to exit a function and return a value to the caller. A function can return multiple values as a tuple.

```
In [ ]: def statistics(numbers):
        """Function to calculate mean and median of a list of numbers."""
        mean = sum(numbers) / len(numbers)
        sorted_numbers = sorted(numbers)
        mid = len(numbers) // 2
        median = (sorted_numbers[mid] + sorted_numbers[~mid]) / 2 # Handles odd/even length
        return mean, median

# Calling the function
data = [10, 20, 30, 40, 50]
mean, median = statistics(data)
print(f"Mean: {mean}, Median: {median}") # Output: Mean: 30.0, Median: 30.0
```

Mean: 30.0, Median: 30.0

SCOPE OF VARIABLES

Variables defined inside a function are local to that function, meaning they cannot be accessed outside it. However, variables defined outside are global and can be accessed by functions unless shadowed by local variables.

```
In [ ]: x = 10 # Global variable

def scope_demo():
    x = 5 # Local variable
    return x

print(scope_demo()) # Output: 5
print(x)            # Output: 10
```

5
10

LAMBDA FUNCTION

Lambda functions are small anonymous functions defined with the lambda keyword. They can take any number of arguments but only have one expression. Lambda functions are often used for short, throwaway functions, especially when used as arguments in higher-order functions like map, filter, or reduce.

```
In [ ]: from functools import reduce

# Using lambda to sum a list of numbers
numbers = [1, 2, 3, 4, 5]
sum_of_numbers = reduce(lambda x, y: x + y, numbers)
print(f"Sum: {sum_of_numbers}") # Output: Sum: 15

# Using lambda with filter to get even numbers
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(f"Even Numbers: {even_numbers}") # Output: Even Numbers: [2, 4]
```

Sum: 15
Even Numbers: [2, 4]

DECORATORS

Decorators are functions that modify the behavior of another function. They are used for logging, enforcing access control, instrumentation, or modifying input/output. A decorator takes a function as an argument and returns a new function that extends its behavior.

```
In [ ]: def logger(func):
        """Decorator to log function calls."""
        def wrapper(*args, **kwargs):
            print(f"Calling function: {func.__name__} with arguments: {args}, {kwargs}")
            result = func(*args, **kwargs)
            print(f"Function {func.__name__} returned: {result}")
            return result
        return wrapper

@logger
def add(a, b):
    """Function to add two numbers."""
    return a + b

# Calling the decorated function
add(5, 7)
```

Calling function: add with arguments: (5, 7), {}
Function add returned: 12

Out[]: 12

RECURSION

Recursion is a programming technique where a function calls itself to solve a problem. Recursive functions must have a base case to terminate and avoid infinite loops.

```
In [ ]: def factorial(n):
        """Recursive function to compute factorial of a number."""
        if n == 0:
            return 1
        else:
            return n * factorial(n - 1)

# Calling the recursive function
print(factorial(5)) # Output: 120
```

120

HIGHER-ORDER FUNCTIONS

Higher-order functions are functions that can take other functions as arguments or return them as results. This allows for greater flexibility and the ability to abstract functionality.

```
In [ ]: def apply_function(func, data):
        """Higher-order function to apply a function to a list of data."""
        return [func(x) for x in data]

# Defining a simple function to double a number
def double(x):
    return x * 2

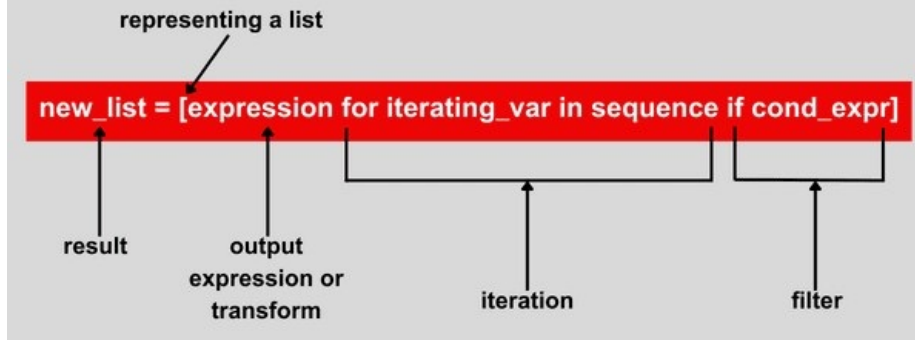
# Using the higher-order function
numbers = [1, 2, 3, 4, 5]
doubled_numbers = apply_function(double, numbers)
print(doubled_numbers) # Output: [2, 4, 6, 8, 10]
```

[2, 4, 6, 8, 10]

LIST COMPREHENSION

List comprehension in Python is a concise way to create lists by applying an expression to each element in an iterable, optionally including conditions to filter elements. It allows for building lists in a single, readable line, making transformations and filtering of data simpler and more efficient compared to traditional loops. The general syntax includes square brackets with an expression followed by a for loop, and optionally an if statement.

Syntax of List Comprehension in Python



```
In [ ]: from itertools import product
```

```
# Sample data
numbers = list(range(1, 16)) # List of numbers from 1 to 15
matrix = [[i + j for j in range(3)] for i in range(1, 10, 3)] # A dynamic 3x3 matrix
words = ["artificial", "intelligence", "machine", "learning", "deep", "neural", "network"]

# Basic list comprehension with embedded logic: Square numbers if odd, cube if even, modulo 3 on all results
complex_numbers = [(x ** 2 if x % 2 != 0 else x ** 3) % 3 for x in numbers]
print("Complex Numbers Transformation:", complex_numbers)

# List comprehension with complex conditionals: Select numbers divisible by 2 or 5, then add 3 if even, subtract 2 if odd
conditional_transformation = [x + 3 if x % 2 == 0 else x - 2 for x in numbers if x % 2 == 0 or x % 5 == 0]
print("Conditional Transformation:", conditional_transformation)

# Nested list comprehension with multi-level filtering: Multiplication matrix with odd rows and even columns only
filtered_multiplication_table = [
    [i * j for j in range(1, 6) if j % 2 == 0] for i in range(1, 6) if i % 2 != 0
]
print("Filtered Multiplication Table:")
for row in filtered_multiplication_table:
    print(row)

# Flattening nested list with conditionals and transformations: Flatten matrix, only add values greater than 4
flattened_filtered_matrix = [num ** 2 for row in matrix for num in row if num > 4]
print("Flattened and Filtered Matrix with Squared Values:", flattened_filtered_matrix)

# Functions in list comprehension with conditions: Length and vowel count of each capitalized word, filter by length
def count_vowels(word):
    return sum(1 for letter in word if letter in 'aeiouAEIOU')

word_analysis = [(word.capitalize(), len(word), count_vowels(word)) for word in words if len(word) > 5]
print("Word Analysis with Vowel Count:", word_analysis)

# Combining multiple comprehensions with Cartesian product: Pair words with numbers, filter by vowel count and number
complex_pairs = [
    (word, num) for word, num in product(words, numbers) if count_vowels(word) % 2 == 0 and num % 3 == 0
]
print("Complex Pairs (Words and Divisible by 3 Numbers):", complex_pairs)

# Ultimate complex example combining all concepts: Triple comprehension with function calls, conditionals, and transformations
ultimate_pairs = [
    (word, num, operation)
    for word in words
    for num in numbers
    for operation in ["add", "subtract"]
    if (count_vowels(word) > 2 and len(word) % 2 == 0)
    and (operation == "add" and num + count_vowels(word) < 20)
    or (operation == "subtract" and num - count_vowels(word) > 0)
]
print("Ultimate Complex Pairs with Operations:", ultimate_pairs)
```

```

Complex Numbers Transformation: [1, 2, 0, 1, 1, 0, 1, 2, 0, 1, 1, 0, 1, 2, 0]
Conditional Transformation: [5, 7, 3, 9, 11, 13, 15, 17, 13]
Filtered Multiplication Table:
[2, 4]
[6, 12]
[10, 20]
Flattened and Filtered Matrix with Squared Values: [25, 36, 49, 64, 81]
Word Analysis with Vowel Count: [('Artificial', 10, 5), ('Intelligence', 12, 5), ('Machine', 7, 3), ('Learning', 8, 3), ('Neural', 6, 3), ('Network', 7, 2)]
Complex Pairs (Words and Divisible by 3 Numbers): [('deep', 3), ('deep', 6), ('deep', 9), ('deep', 12), ('deep', 15), ('network', 3), ('network', 6), ('network', 9), ('network', 12), ('network', 15)]
Ultimate Complex Pairs with Operations: [('artificial', 1, 'add'), ('artificial', 2, 'add'), ('artificial', 3, 'add'), ('artificial', 4, 'add'), ('artificial', 5, 'add'), ('artificial', 6, 'add'), ('artificial', 6, 'subtract'), ('artificial', 7, 'add'), ('artificial', 7, 'subtract'), ('artificial', 8, 'add'), ('artificial', 8, 'subtract'), ('artificial', 9, 'add'), ('artificial', 9, 'subtract'), ('artificial', 10, 'add'), ('artificial', 10, 'subtract'), ('artificial', 11, 'add'), ('artificial', 11, 'subtract'), ('artificial', 12, 'add'), ('artificial', 12, 'subtract'), ('artificial', 13, 'add'), ('artificial', 13, 'subtract'), ('artificial', 14, 'add'), ('artificial', 14, 'subtract'), ('artificial', 15, 'subtract'), ('intelligence', 1, 'add'), ('intelligence', 2, 'add'), ('intelligence', 3, 'add'), ('intelligence', 4, 'add'), ('intelligence', 5, 'add'), ('intelligence', 6, 'add'), ('intelligence', 6, 'subtract'), ('intelligence', 7, 'add'), ('intelligence', 7, 'subtract'), ('intelligence', 8, 'add'), ('intelligence', 8, 'subtract'), ('intelligence', 9, 'add'), ('intelligence', 9, 'subtract'), ('intelligence', 10, 'add'), ('intelligence', 10, 'subtract'), ('intelligence', 11, 'add'), ('intelligence', 11, 'subtract'), ('intelligence', 12, 'add'), ('intelligence', 12, 'subtract'), ('intelligence', 13, 'add'), ('intelligence', 13, 'subtract'), ('intelligence', 14, 'add'), ('intelligence', 14, 'subtract'), ('intelligence', 15, 'subtract'), ('machine', 4, 'subtract'), ('machine', 5, 'subtract'), ('machine', 6, 'subtract'), ('machine', 7, 'subtract'), ('machine', 8, 'subtract'), ('machine', 9, 'subtract'), ('machine', 10, 'subtract'), ('machine', 11, 'subtract'), ('machine', 12, 'subtract'), ('machine', 13, 'subtract'), ('machine', 14, 'subtract'), ('machine', 15, 'subtract'), ('learning', 1, 'add'), ('learning', 2, 'add'), ('learning', 3, 'add'), ('learning', 4, 'add'), ('learning', 4, 'subtract'), ('learning', 5, 'add'), ('learning', 5, 'subtract'), ('learning', 6, 'add'), ('learning', 6, 'subtract'), ('learning', 7, 'add'), ('learning', 7, 'subtract'), ('learning', 8, 'add'), ('learning', 8, 'subtract'), ('learning', 9, 'add'), ('learning', 9, 'subtract'), ('learning', 10, 'add'), ('learning', 10, 'subtract'), ('learning', 11, 'add'), ('learning', 11, 'subtract'), ('learning', 12, 'add'), ('learning', 12, 'subtract'), ('learning', 13, 'add'), ('learning', 13, 'subtract'), ('learning', 14, 'add'), ('learning', 14, 'subtract'), ('learning', 15, 'add'), ('learning', 15, 'subtract'), ('deep', 3, 'subtract'), ('deep', 4, 'subtract'), ('deep', 5, 'subtract'), ('deep', 6, 'subtract'), ('deep', 7, 'subtract'), ('deep', 8, 'subtract'), ('deep', 9, 'subtract'), ('deep', 10, 'subtract'), ('deep', 11, 'subtract'), ('deep', 12, 'subtract'), ('deep', 13, 'subtract'), ('deep', 14, 'subtract'), ('deep', 15, 'subtract'), ('neural', 1, 'add'), ('neural', 2, 'add'), ('neural', 3, 'add'), ('neural', 4, 'add'), ('neural', 4, 'subtract'), ('neural', 5, 'add'), ('neural', 5, 'subtract'), ('neural', 6, 'add'), ('neural', 6, 'subtract'), ('neural', 7, 'add'), ('neural', 7, 'subtract'), ('neural', 8, 'add'), ('neural', 8, 'subtract'), ('neural', 9, 'add'), ('neural', 9, 'subtract'), ('neural', 10, 'add'), ('neural', 10, 'subtract'), ('neural', 11, 'add'), ('neural', 11, 'subtract'), ('neural', 12, 'add'), ('neural', 12, 'subtract'), ('neural', 13, 'add'), ('neural', 13, 'subtract'), ('neural', 14, 'add'), ('neural', 14, 'subtract'), ('neural', 15, 'add'), ('neural', 15, 'subtract'), ('network', 3, 'subtract'), ('network', 4, 'subtract'), ('network', 5, 'subtract'), ('network', 6, 'subtract'), ('network', 7, 'subtract'), ('network', 8, 'subtract'), ('network', 9, 'subtract'), ('network', 10, 'subtract'), ('network', 11, 'subtract'), ('network', 12, 'subtract'), ('network', 13, 'subtract'), ('network', 14, 'subtract'), ('network', 15, 'subtract')]

```

The above Python program demonstrates intricate list comprehensions, including conditional logic, nested structures, and custom functions. It performs complex transformations, filtering, and flattening on sample data, showing multiple levels of nested comprehensions and function calls. The code is designed to illustrate high-level comprehension techniques for sophisticated data manipulation.

BASICS OF LIST COMPREHENSION

Basic list comprehension consists of an expression followed by a for loop within square brackets. This structure generates a list by applying the expression to each item in the iterable.

```

In [ ]: # Generate a list of squares for numbers from 1 to 10
squares = [x**2 for x in range(1, 11)]
print(squares) # Output: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

```

LIST COMPREHENSION WITH CONDITIONALS

You can add conditional expressions to filter items in a list comprehension, enabling you to include elements that meet specific criteria.

```

In [ ]: # Generate a list of even squares for numbers from 1 to 10
even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]
print(even_squares) # Output: [4, 16, 36, 64, 100]

[4, 16, 36, 64, 100]

```

NESTED LIST COMPREHENSION

Nested list comprehension involves multiple for loops, often used to create matrices or manipulate elements in nested structures. This structure allows for generating lists with more complex relationships.

```
In [ ]: # Generate a 3x3 matrix with list comprehension
matrix = [[i * j for j in range(1, 4)] for i in range(1, 4)]
print(matrix) # Output: [[1, 2, 3], [2, 4, 6], [3, 6, 9]]

[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

FLATTENING NESTED LISTS

List comprehension can be used to flatten a nested list into a single list by combining multiple loops. This is especially useful for working with multidimensional lists or arrays.

```
In [ ]: # Flatten a 2D list into a single list
nested_list = [[1, 2, 3], [4, 5], [6, 7, 8]]
flat_list = [item for sublist in nested_list for item in sublist]
print(flat_list) # Output: [1, 2, 3, 4, 5, 6, 7, 8]

[1, 2, 3, 4, 5, 6, 7, 8]
```

FUNCTIONS IN LIST COMPREHENSION

List comprehension can call functions on each item, making it possible to apply transformations that require more complex logic than an expression alone.

```
In [ ]: # Define a function to double a number
def double(x):
    return x * 2

# Use the function in list comprehension to double each number in the range
doubled_numbers = [double(x) for x in range(1, 6)]
print(doubled_numbers) # Output: [2, 4, 6, 8, 10]

[2, 4, 6, 8, 10]
```

COMBINING MULTIPLE LIST COMPREHENSIONS

The chain multiple list comprehensions to apply sequential transformations to a list. This approach can enhance code readability by breaking down complex transformations.

```
In [ ]: # First, filter odd numbers and then square them
odd_squares = [x**2 for x in [y for y in range(1, 11) if y % 2 != 0]]
print(odd_squares) # Output: [1, 9, 25, 49, 81]

[1, 9, 25, 49, 81]
```

ERROR HANDLING

Error handling in Python is the process of responding to and managing exceptions, or unexpected conditions that arise during the execution of a program. By using specific statements to catch and handle errors, Python allows for robust, user-friendly applications that can anticipate issues and respond gracefully.

```
In [ ]: from math import sqrt, log

# Custom exceptions for various scenarios
class NegativeNumberError(Exception):
    def __init__(self, message="Negative number encountered in processing"):
        self.message = message
        super().__init__(self.message)

class InsufficientDataError(Exception):
    def __init__(self, message="Insufficient data provided for operation"):
        self.message = message
        super().__init__(self.message)

# Complex data processing function with advanced exception handling
def complex_data_processing(data):
    try:
        if not isinstance(data, list) or len(data) < 2:
            raise InsufficientDataError("Data must be a list with at least two elements.")

        # compute the division of the first two elements
```

```

    try:
        result = data[0] / data[1]
    except ZeroDivisionError as e:
        print(f"Error: Cannot divide by zero. {e}")
        return None
    except TypeError as e:
        print(f"Type Error: Non-numeric data encountered. {e}")
        return None
    else:
        print("Division successful.")

    # calculate the square root if result is non-negative
    try:
        if result < 0:
            raise NegativeNumberError("Square root of a negative number.")
        sqrt_result = sqrt(result)
    except NegativeNumberError as e:
        print(f"Custom Error: {e}")
        return None
    else:
        print(f"Square root of result: {sqrt_result}")

    # calculate the logarithm if result is positive
    try:
        if result <= 0:
            raise ValueError("Logarithm of zero or negative number.")
        log_result = log(result)
    except ValueError as e:
        print(f"Value Error: {e}")
        return None
    else:
        print(f"Logarithm of result: {log_result}")

    finally:
        print("Data processing complete. Releasing any acquired resources.")

    # Return both results if all operations were successful
    return {"sqrt_result": sqrt_result, "log_result": log_result}

# Main function to handle user input and raise custom exceptions if necessary
def main():
    try:
        # Simulate complex input with user or predefined data
        raw_input = input("Enter a list of at least two numbers (e.g., [4, 2]): ")
        try:
            data = eval(raw_input) # WARNING: Only for demonstration purposes. Avoid using `eval` on untrusted
            if not all(isinstance(i, (int, float)) for i in data):
                raise TypeError("All elements must be numbers.")
        except SyntaxError:
            print("Input Error: Invalid list format.")
            return
        except TypeError as e:
            print(f"Data Error: {e}")
            return

        # Run the complex processing with advanced error handling
        results = complex_data_processing(data)
        if results:
            print("Final Results:", results)

    except KeyboardInterrupt:
        print("Process interrupted by user.")
    finally:
        print("End of program execution. Cleaning up any resources.")

# Run the main function
main()

```

```

Enter a list of at least two numbers (e.g., [4, 2]): [5,6]
Division successful.
Square root of result: 0.9128709291752769
Logarithm of result: -0.1823215567939546
Data processing complete. Releasing any acquired resources.
Final Results: {'sqrt_result': 0.9128709291752769, 'log_result': -0.1823215567939546}
End of program execution. Cleaning up any resources.

```

This Python program demonstrates complex exception handling by performing multi-step data processing with nested try-except blocks, custom exceptions, and data validation. It handles division, square root, and logarithmic operations with detailed error messages for each potential issue. The program includes custom error classes and ensures cleanup with finally blocks, enhancing robustness and clarity.

TRY AND EXCEPT BLOCK

The try and except block is the primary structure for handling errors in Python. Code that may cause an exception is placed in the try block, and if an exception occurs, the except block catches and handles it.

```
In [ ]: # Division with error handling for division by zero
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(f"Result: {result}")
except ZeroDivisionError as e:
    print("Error: Cannot divide by zero.", e)
```

Enter a number: 2
Result: 5.0

HANDLING MULTIPLE EXCEPTIONS

Python allows you to handle multiple exception types in a single try block by chaining except blocks. This helps handle different types of errors separately, providing more specific responses.

```
In [ ]: # Handling ValueError and ZeroDivisionError
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("Error: Invalid input. Please enter a valid integer.")
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
else:
    print(f"Result: {result}")
```

Enter a number: 2
Result: 5.0

USING ELSE BLOCK

The else block runs only if the try block executes without any exceptions. It's useful for code that should only run when no errors are encountered.

```
In [ ]: # Using else block to confirm successful execution
try:
    with open("data.txt", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("Error: File not found.")
else:
    print("File read successfully:")
    print(data)
```

Error: File not found.

FINALLY BLOCK

The finally block runs after the try and except blocks, regardless of whether an exception was raised. It is commonly used for cleanup actions, like closing files or releasing resources.

```
In [ ]: # Using finally block for resource cleanup
try:
    file = open("data.txt", "r")
    data = file.read()
    print(data)
except FileNotFoundError:
    print("Error: File not found.")
finally:
    if 'file' in locals() and not file.closed:
        file.close()
    print("File closed successfully.")
```

Error: File not found.

RAISING EXCEPTIONS WITH RAISE

The raise statement allows you to manually throw an exception, which is helpful for enforcing

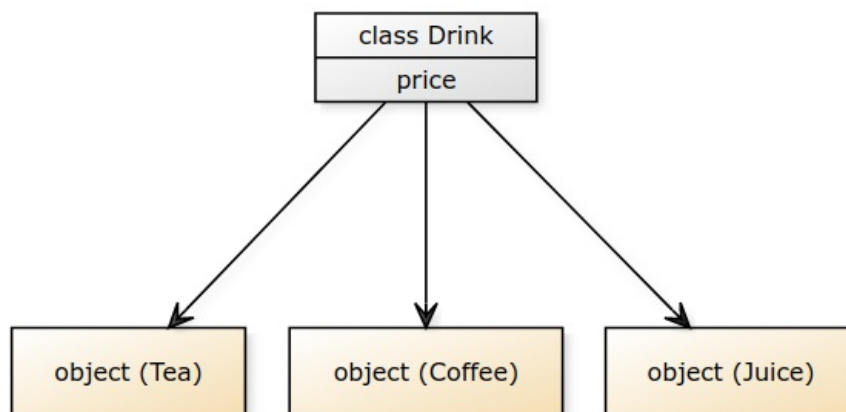
conditions or validating data within functions.

```
In [ ]: def set_age(age):  
    if age < 0:  
        raise ValueError("Age cannot be negative.")  
    print("Age is valid.")  
  
    try:  
        set_age(-5)  
    except ValueError as e:  
        print("Error:", e)
```

Error: Age cannot be negative.

CLASSES AND OBJECT

1. **Class as a Template:** A class defines the structure and behavior for creating objects, outlining the attributes and methods they will have.
2. **Object as a Concrete Instance:** An object is a specific instance of a class, holding unique data while sharing the class's methods.
3. **Defining Attributes and Behaviors:** Classes contain attributes (data) and methods (functions), determining what an object knows and how it operates.
4. **Encapsulation of Data and Functionality:** Classes encapsulate both data and behavior, ensuring that access to internal details is controlled through methods.
5. **Reusability and Inheritance:** Classes allow for code reuse and enable inheritance, where new classes can inherit attributes and methods from existing ones, promoting efficiency.



EXAMPLE PROGRAM

```
In [1]: # Base class: Car  
class Car:  
    def __init__(self, model, color):  
        self.__model = model # Private attribute  
        self.color = color # Public attribute  
  
    # Method to start the engine  
    def start_engine(self):  
        print(f"The {self.__model} engine is now running.")  
  
    # Getter method to access the private attribute __model  
    def get_model(self):  
        return self.__model  
  
    # Method to display car details  
    def display_info(self):  
        print(f"Car Model: {self.__model}")  
        print(f"Car Color: {self.color}")  
  
# Derived class: ElectricCar inherits from Car  
class ElectricCar(Car):  
    def __init__(self, model, color, battery_size):  
        super().__init__(model, color) # Calling the parent class's __init__ method  
        self.battery_size = battery_size # Additional attribute for ElectricCar
```

```

# Overriding the start_engine method to implement polymorphism
def start_engine(self):
    print(f"The {self.get_model()} electric engine is silently running.")

# Method to display electric car details
def display_info(self):
    super().display_info() # Calling parent class's display_info
    print(f"Battery Size: {self.battery_size} kWh")

# Creating an object of Car class
my_car = Car("Toyota", "Red")
my_car.start_engine() # Output: The Toyota engine is now running.
my_car.display_info() # Output: Car Model: Toyota, Car Color: Red

# Creating an object of ElectricCar class
my_electric_car = ElectricCar("Tesla Model 3", "Blue", 75)
my_electric_car.start_engine() # Output: The Tesla Model 3 electric engine is silently running.
my_electric_car.display_info() # Output: Car Model: Tesla Model 3, Car Color: Blue, Battery Size: 75 kWh

```

The Toyota engine is now running.

Car Model: Toyota

Car Color: Red

The Tesla Model 3 electric engine is silently running.

Car Model: Tesla Model 3

Car Color: Blue

Battery Size: 75 kWh

Class Definition: The Car class is the base class, and the ElectricCar class is a subclass that inherits from Car.

Object: Objects like my_car and my_electric_car are instances of the Car and ElectricCar classes, representing specific cars. Each object holds its own set of attributes and behaviors.

Instance Attributes: In the Car class, model and color are instance attributes. The model attribute is private (prefixed with __), while color is public.

Methods: The Car class has methods such as start_engine, display_info, and get_model. The ElectricCar class overrides the start_engine method to demonstrate polymorphism, where the method behaves differently in the subclass.

Encapsulation: The model attribute is private in the Car class, and can only be accessed through the get_model() method.

Inheritance and Polymorphism: The ElectricCar class inherits from Car, and the start_engine method is overridden to show different behavior (silently running engine in the electric car). The display_info method is also extended to include the battery size for electric cars.

HANDS-ON PYTHON CHALLENGES

1. Write a Python program to process a large dataset (list of dictionaries) containing user data. Use list comprehensions to filter users over 18, transform their ages into months, and return the top 5 youngest users in the dataset. Ensure the program handles potential missing or malformed data gracefully using try-except blocks.

Hint: Use list comprehension to filter users over the age of 18. Use a try-except block to catch any errors like KeyError or TypeError when processing user data. Example: You can filter users by checking if 'age' in user and then use user['age'] > 18.

2. Create a function that accepts a list of numbers and returns the result of dividing each number by its corresponding index. Handle any potential ZeroDivisionError using a try-except block, and implement a finally block to ensure resources are cleaned up, such as closing any open files.

Hint: Write a function that takes a list of numbers and divides each number by its index in the list. Handle ZeroDivisionError using try-except. Use a finally block to close any opened resources (e.g., files or network connections).

3. Design a Python program that parses a CSV file containing student records (name, age, grade). Create a custom exception to handle errors when a student record is missing required data. Use recursion to count the number of students with grades above a certain threshold and print the result.

Hint: Use the csv module to parse a CSV file. Handle missing data with a try-except block, and raise a custom exception if needed. Use recursion to count students with grades above a certain threshold (e.g., count recursively through the list).

4. Implement a Python function that accepts a string and returns a dictionary containing the count of each character in the string. Use a lambda function to count the occurrences of specific vowels only. Optimize the solution using list comprehension and handle any unexpected characters with a custom exception.

Hint: Write a function that uses `collections.Counter` or a dictionary to count characters. Use a lambda function to filter vowels (e.g., `lambda x: x in 'aeiou'`). Handle invalid characters using a custom exception or simple error checking.

5. Create a Python class that represents a bank account. The class should have methods to deposit, withdraw, and check balance. Handle exceptions such as insufficient funds and invalid deposit amounts using multiple `except` blocks. Add a feature to automatically close the account after 3 failed login attempts, with a final statement to clean up resources.

Hint: Define methods for `deposit()`, `withdraw()`, and `check_balance()`. Handle insufficient funds using a `ValueError`. Use a `finally` block to close any resources or print a summary message.

6. Build a Python program that reads and processes a list of employee data. The program should use nested loops and list comprehension to categorize employees based on age and department. Handle cases where the data is malformed, and raise custom exceptions for any invalid department or age values.

Hint: Use list comprehensions to categorize employees by age or department. Nested loops can be useful for going through multiple lists (e.g., employees in different departments). Handle errors when data is missing using a `try-except` block.

7. Design a Python program that accepts a list of integers and returns the sum of all numbers that are prime, using a lambda function for the primality check. Use list comprehension for filtering prime numbers and include a `try-except` block to handle any errors during calculation.

Hint: Use a lambda function to check for prime numbers (e.g., `lambda x: all(x % i != 0 for i in range(2, int(x**0.5)+1))`). Use list comprehension to filter and sum prime numbers. Handle non-integer inputs with a `try-except` block.

8. Write a function to implement a simple calculator that can add, subtract, multiply, and divide. Use the conditional (ternary) operator to return results in a single line. Ensure that invalid operations (like dividing by zero) raise a custom exception and are handled correctly.

Hint: Use basic arithmetic operators (+, -, *, /). Implement the ternary operator to return results in one line: `result = "Invalid" if divisor == 0 else dividend / divisor`. Catch division by zero with a `try-except` block.

9. Create a Python program to read a JSON file containing product data. Use recursion to navigate through nested structures and extract specific product details. Handle potential file I/O errors using `try-except` blocks and ensure that all resources are properly closed in the `finally` block.

Hint: Use the `json` module to load data from a JSON file. Recursively traverse nested structures (e.g., for products, prices). Use `try-except` to handle errors like file not found.

10. Implement a Python program that accepts two lists of equal length and performs element-wise addition using a list comprehension. If an element is not a valid number (e.g., a string or `None`), raise an exception and provide an error message with the `raise` keyword.

Hint: Use list comprehension to add corresponding elements of two lists. Check if both lists are the same length before proceeding. Use `try-except` to handle type errors if one list contains non-numeric data.

11. Write a Python program that implements a simple command-line to-do list application. The user should be able to add tasks, mark tasks as complete, and delete tasks. Ensure that all actions are logged to a file, using exception handling for invalid file operations and adding a `finally` block to ensure the file is always closed.

Hint: Create a simple menu with options to add, complete, and delete tasks. Use file operations to log the actions to a text file. Ensure file is properly closed using a `finally` block.

12. Design a Python script that uses decorators to cache the result of an expensive function that calculates factorials. The function should be optimized to prevent redundant calculations, and error handling should be implemented to handle large inputs or invalid data gracefully.

Hint: Write a decorator function to cache the result of a factorial function (e.g., store results in a dictionary). Use the decorator to optimize repeated factorial calculations. Handle large numbers gracefully and provide error handling for invalid input.

